

DiffusionGemma — Öğrenme

Dokümantasyonu

Saf PyTorch implementasyonu · Türkçe · Mermaid diyagramları · LaTeX formüller

Tarih: 2026-06-13 · [paper-implementations/diffusionGemma](#)

İçindekiler

- Genel Bakış
- Temel Matematik
- `configuration.py`
- `generation.py`
- `modeling.py`
- Öğrenme Rehberi

1. Genel Bakış

Otoregresif LLM'ler soldan sağa koşullu dağılım örnekler:

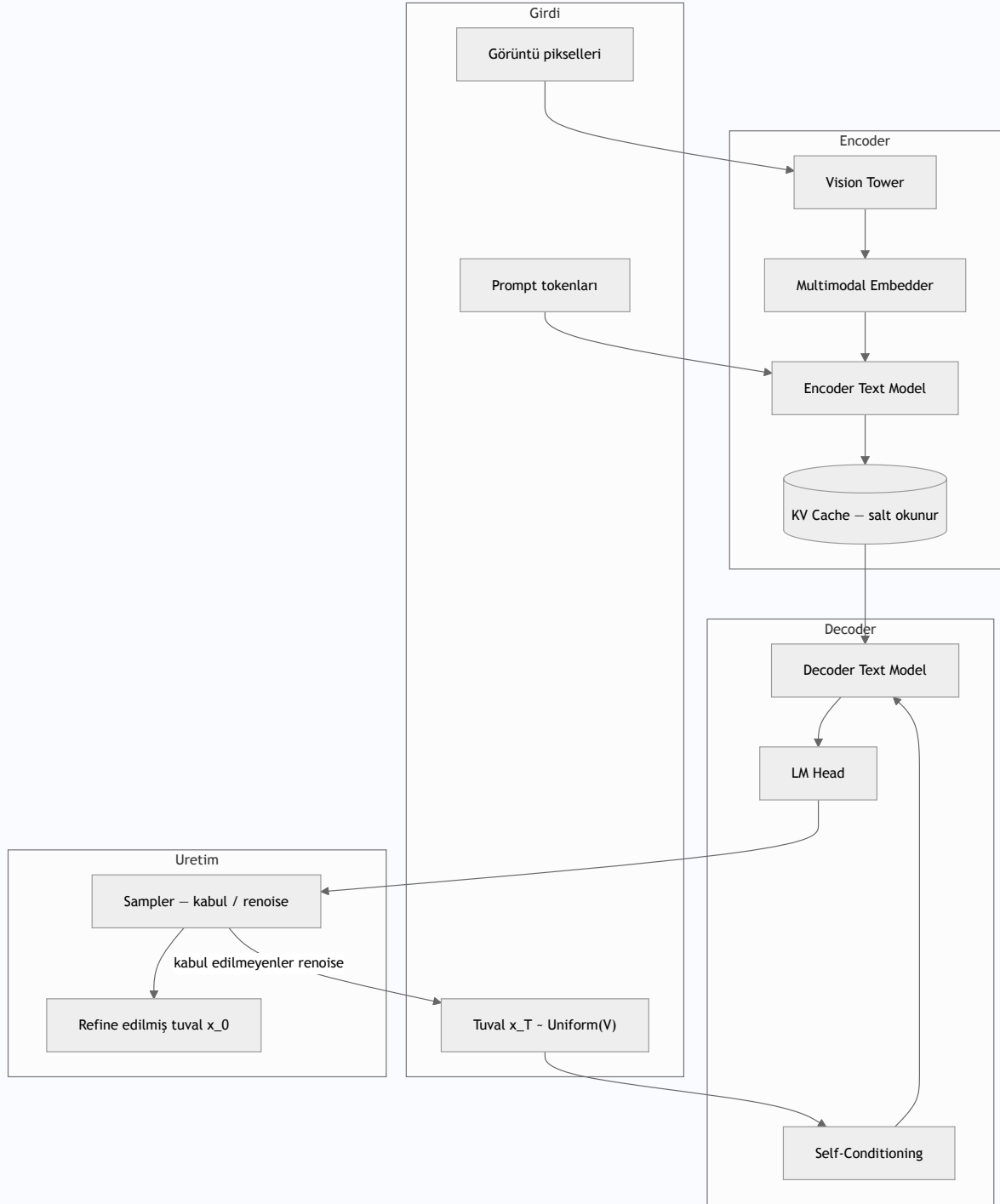
$P(x_t \mid x_{<t})$

Difüzyon tabanlı DiffusionGemma ise sabit uzunlukta bir **tuval** (canvas) üzerinde iteratif gürültü giderme yapar:

$$x_T \sim \text{Uniform}(\mathcal{V}) \xrightarrow{\text{denoise}} x_0$$

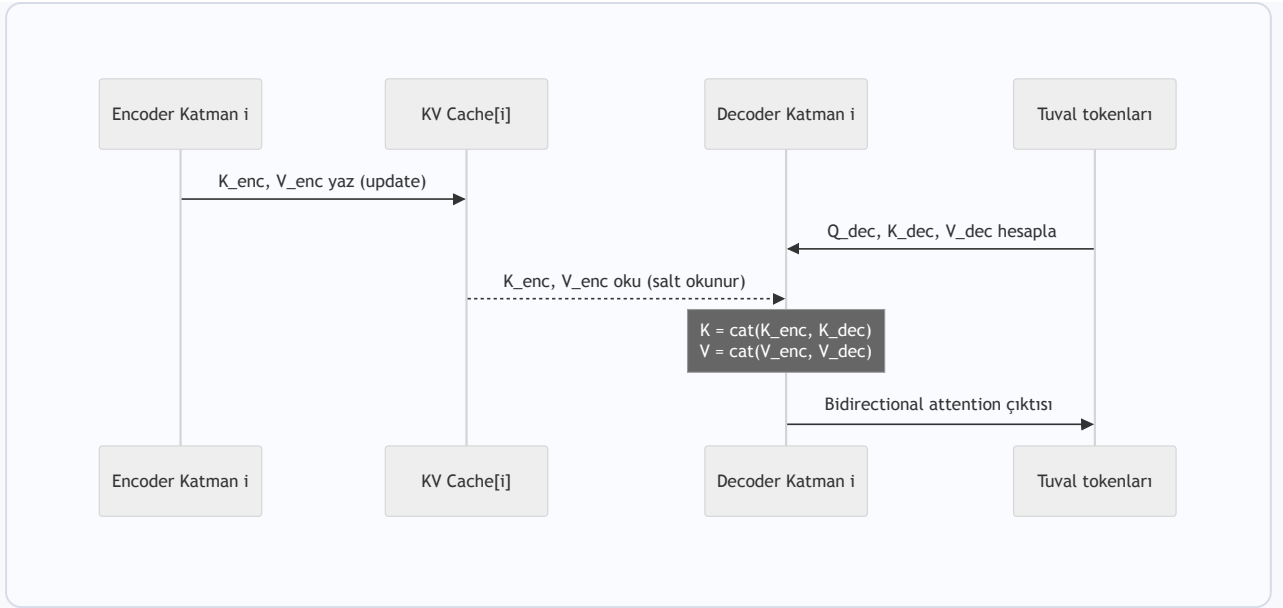
Her adımda model tuvalin tamamına bakarak logits üretir; sampler bir kısmını kabul eder, geri kalanı yeniden gürültüler.

1.1 Mimari veri akışı



1.2 Encoder–Decoder ilişkisi

Klasik seq2seq modellerde decoder, encoder'ın *son gizli durumunu* okur. DiffusionGemma'da decoder, encoder'ın *her katmanındaki K/V önbelleğini* salt okunur kullanır:



1.3 Dosya yapısı

- `configuration.py` — hiperparametreler
- `modeling.py` — tüm nn.Module sınıfları
- `generation.py` — difüzyon sampling

2. Temel Matematik

2.1 RMSNorm

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}} \odot \gamma$$

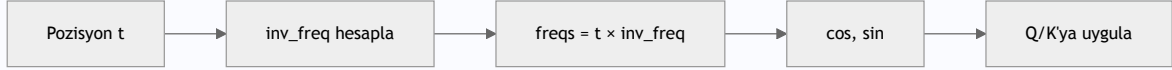
LayerNorm'dan farklı olarak ortalama çıkarılmaz; Gemma ailesinin standart normalizasyonudur.

2.2 RoPE (Rotary Positional Embedding)

$$\text{inv_freq}_i = \frac{1}{\theta^{2i/d}}, \quad \theta = \text{rope_theta}$$

$$\text{freqs}_{t,i} = t \cdot \text{inv_freq}_i, \quad \text{emb} = [\cos(\text{freqs}), \sin(\text{freqs})]$$

$$x_{\text{rot}} = x \odot \cos + \text{rotate_half}(x) \odot \sin$$



2.3 Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{s} + M\right) V$$

DiffusionGemma'da $s = 1$ (Gemma scaling; klasik $1/\sqrt{d_k}$ değil). M additive mask'tir.

2.4 Grouped Query Attention (GQA)

$$n_{\text{rep}} = \frac{n_{\text{heads}}}{n_{\text{kv_heads}}}, \quad K' = \text{repeat_kv}(K), \quad V' = \text{repeat_kv}(V)$$

2.5 SwiGLU FFN (Gemma MLP)

$$\text{FFN}(x) = W_{\downarrow}(\sigma(W_{\text{gate}}x) \odot W_{\uparrow}x)$$

2.6 MoE Router

$$p_e = \text{softmax}(W_r \cdot \text{norm}(x))_e, \quad \mathcal{E}_t = \text{topk}(p, k)$$

$$y = \sum_{e \in \mathcal{E}_t} \frac{p_e}{\sum_{e'} p_{e'}} \cdot \text{Expert}_e(x)$$

2.7 Logits Softcapping

$$\ell' = s \cdot \tanh\left(\frac{\ell}{s}\right), \quad s = 30$$

2.8 Self-Conditioning Soft Embedding

$$\tilde{e} = \alpha \cdot \text{softmax}(\ell) \cdot E, \quad \alpha = \sqrt{d_{\text{hidden}}}$$

Önceki adım logits'inden elde edilen yumuşak gömme, giriş embedding'ine eklenir.

2.9 Sıcaklık programı

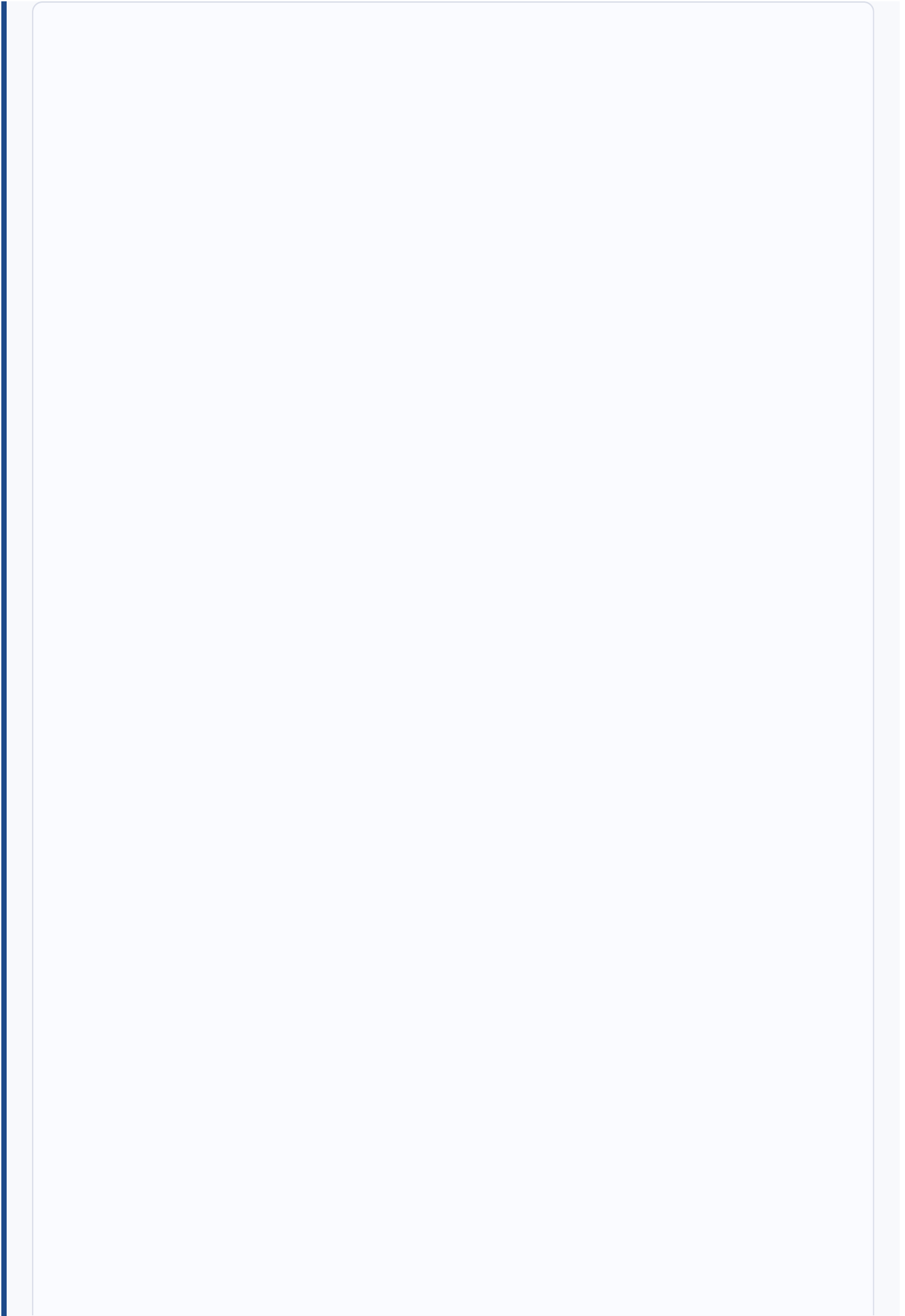
$$t(n) = t_{\min} + (t_{\max} - t_{\min}) \cdot \frac{n}{N}, \quad \ell_{\text{scaled}} = \frac{\ell}{t(n)}$$

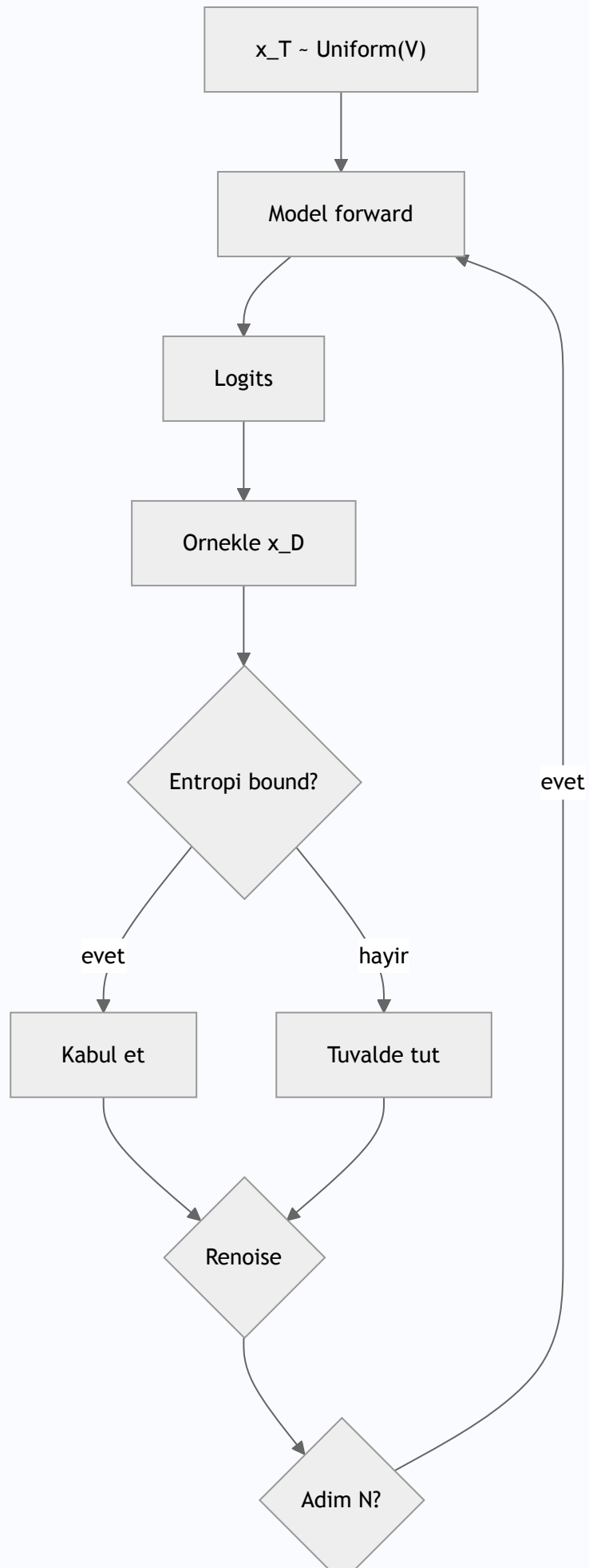
2.10 Entropi sınırı (EntropyBoundSampler)

$$H_i = - \sum_v p_i(v) \log p_i(v)$$

$$\sum_{i=1}^k H_{(i)} - \max_{1 \leq i \leq k} H_{(i)} \leq B \implies \text{k token kabul}$$

$H_{(i)}$ entropiye göre artan sırada; B = entropy_bound. Bu, kabul edilen tokenların yaklaşık bağımsızlığını sağlar.





hayir

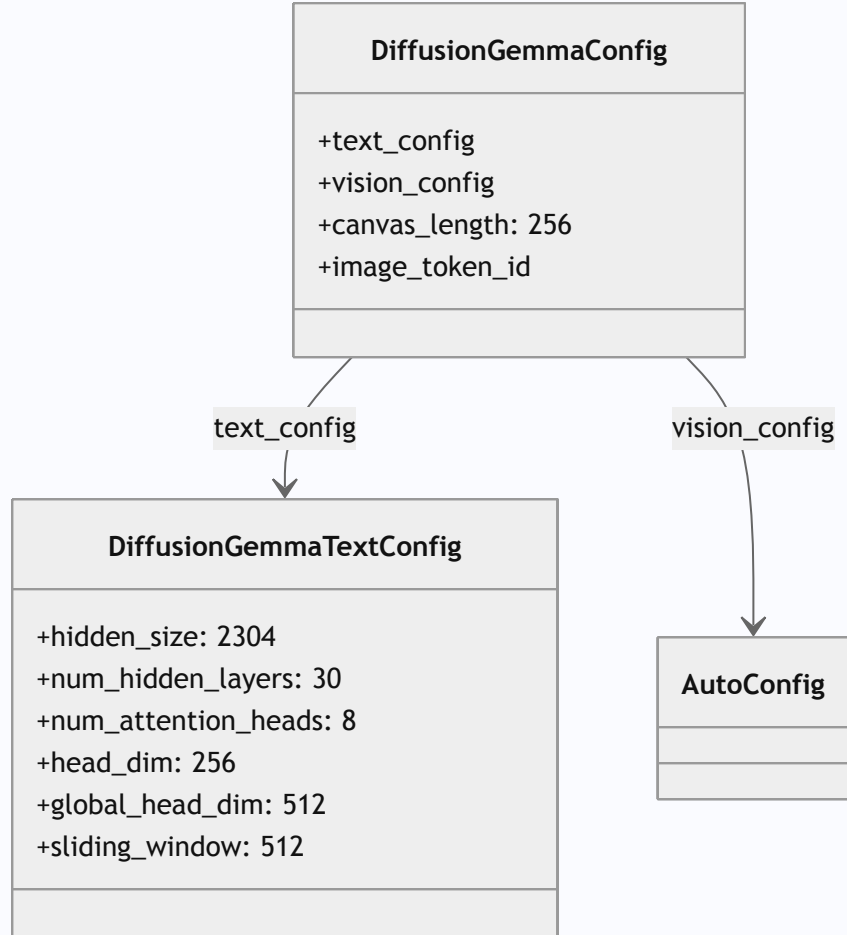
x_0 final tuval

3. configuration.py

DiffusionGemma model hiperparametreleri.

Bu modül, HuggingFace `configuration\_diffusion\_gemma.py` dosyasının öğrenme amaçlı sadeleştirilmiş halidir. Sınıf değişkenleri doğrudan varsayılan checkpoint değerlerini taşır; ayrı bir `__init__` yoktur.

Öğrenciler için ipucu: Bu sayılar modelin bellek kullanımını, hızını ve kapasitesini belirler. Örneğin `hidden_size=2304`, her token için 2304 boyutlu bir vektör anlamına gelir.



DiffusionGemmaTextConfig

Metin gövdesi (encoder + decoder ortak text backbone) yapılandırması.

DiffusionGemma'nın metin tarafı Gemma 4 mimarisine dayanır. Katmanların bir kısmı sliding window attention (yerel, 512 token pencere), son katman(lar) ise full/global attention (tüm diziye bakış, daha büyük head) kullanır.

Önemli kavramlar

GQA (Grouped Query Attention):

num_attention_heads=8 sorgu head'i, num_key_value_heads=4 K/V head'i.

Her K/V head'i 2 Q head'ine hizmet eder → bellek tasarrufu.

Sliding vs Full attention:

sliding: head_dim=256, pencere=512 — uzun dizilerde verimli

full: global_head_dim=512 — geniş bağlam, daha fazla parametre

use_bidirectional_attention:

"all" → encoder'da tüm tokenlar birbirini görür (difüzyon eğitimi)

"vision" → görüntü tokenları çift yönlü, metin causal kalır

None → varsayılan davranış (genelde causal)

MoE (Mixture of Experts):

top_k_experts kadar expert her token için aktif edilir. Router skorları

hangi expert'lerin çalışacağını belirler; kalan expert'ler atlanır.

Kaynak — satır 16

```
16 | class DiffusionGemmaTextConfig:
17 |     """
18 |     Metin gövdesi (encoder + decoder ortak text backbone) yapılandırması.
19 |
20 |     DiffusionGemma'nın metin tarafı Gemma 4 mimarisine dayanır. Katmanların
21 |     bir kısmı sliding window attention (yerel, 512 token pencere), son
22 |     katman(lar) ise full/global attention (tüm diziye bakış, daha büyük head)
23 |     kullanır.
24 |
25 |     Önemli kavramlar
26 |     -----
27 |     GQA (Grouped Query Attention):
28 |         num_attention_heads=8 sorgu head'i, num_key_value_heads=4 K/V head'i.
29 |         Her K/V head'i 2 Q head'ine hizmet eder → bellek tasarrufu.
30 |
31 |     Sliding vs Full attention:
32 |         sliding: head_dim=256, pencere=512 – uzun dizilerde verimli
33 |         full: global_head_dim=512 – geniş bağlam, daha fazla parametre
34 |
35 |     use_bidirectional_attention:
36 |         "all" → encoder'da tüm tokenlar birbirini görür (difüzyon eğitimi)
37 |         "vision" → görüntü tokenları çift yönlü, metin causal kalır
38 |         None → varsayılan davranış (genelde causal)
39 |
40 |     MoE (Mixture of Experts):
41 |         top_k_experts kadar expert her token için aktif edilir. Router skorları
42 |         hangi expert'lerin çalışacağını belirler; kalan expert'ler atlanır.
43 |     """
44 |
45 |     # --- Kelime dağarcığı ve gömme ---
```

DiffusionGemmaConfig

Tam DiffusionGemma yapılandırması: metin + görüntü + difüzyon tuvali.

Mimari özeti

```
| DiffusionGemmaConfig |  
| └── text_config → DiffusionGemmaTextConfig |  
| └── vision_config → görüntü encoder (SigLIP vb.) |  
| └── canvas_length → difüzyon tuvali token sayısı |
```

Çok modlu akış:

1. Görüntü → vision_tower → embed_vision → metin uzayına projeksiyon
2. Metin + görüntü tokenları encoder'da işlenir → KV cache
3. canvas_length boyutunda gürültülü tuval decoder'da refine edilir

Özel görüntü tokenları:

boi_token_id / eoi_token_id: görüntü bloğunun sınırları

image_token_id: placeholder; gerçek görüntü embedding'i ile değiştirilir

Kaynak — satır 139

```
139 | class DiffusionGemmaConfig:
140 |     """
141 |     Tam DiffusionGemma yapılandırması: metin + görüntü + difüzyon tuvali.
142 |
143 |     Mimari Özeti
144 |     -----
145 |
146 |     DiffusionGemmaConfig
147 |     └─ text_config    → DiffusionGemmaTextConfig
148 |     └─ vision_config → görüntü encoder (SigLIP vb.)
149 |     └─ canvas_length → difüzyon tuvali token sayısı
150 |
151 |
152 |     Çok modlu akış:
153 |     1. Görüntü → vision_tower → embed_vision → metin uzayına projeksiyon
154 |     2. Metin + görüntü tokenları encoder'da işlenir → KV cache
155 |     3. canvas_length boyutunda gürültülü tuval decoder'da refine edilir
156 |
157 |     Özel görüntü tokenları:
158 |     boi_token_id / eoi_token_id: görüntü bloğunun sınırları
159 |     image_token_id: placeholder; gerçek görüntü embedding'i ile değiştirilir
160 |     """
161 |
162 |     sub_configs = {
163 |         "text_config": DiffusionGemmaTextConfig,
164 |         "vision_config": AutoConfig,
165 |     }
166 |     text_config = None
167 |     vision_config = None
168 |
```

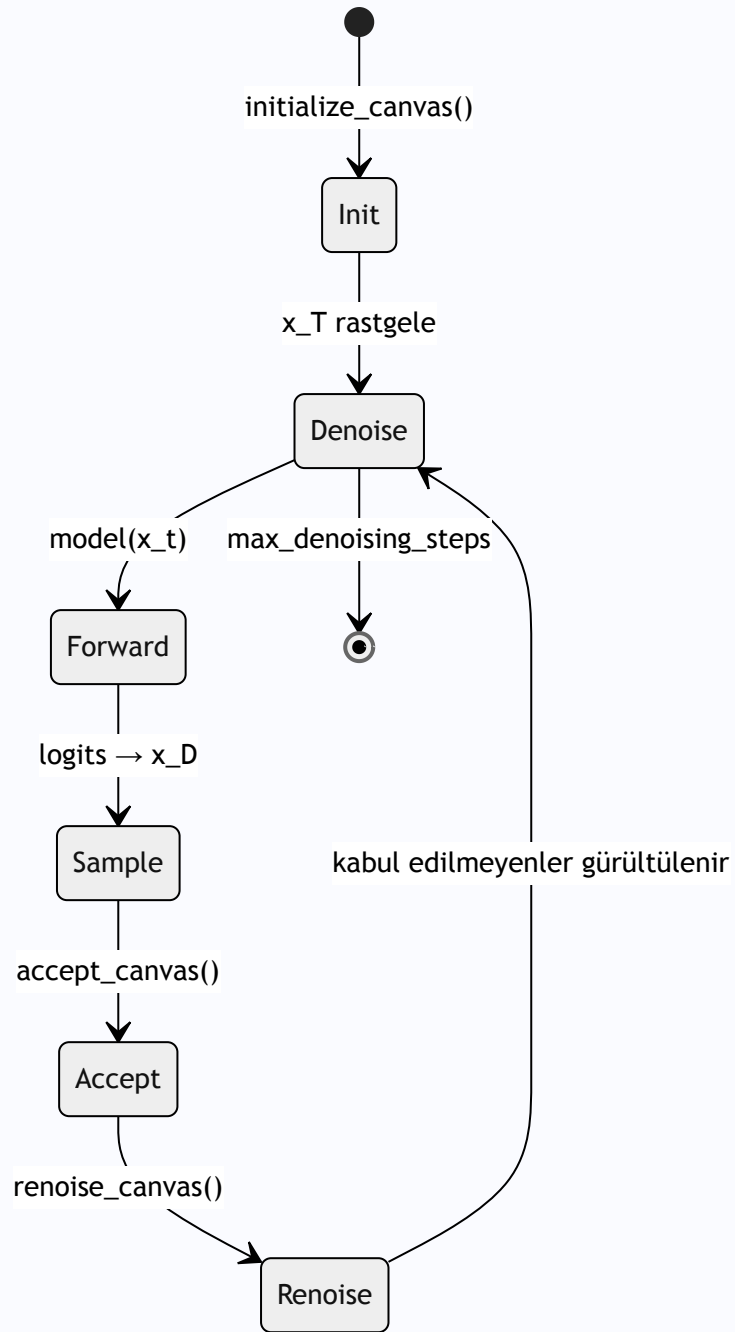
4. generation.py

DiffusionGemma metin üretimi — difüzyon tabanlı sampling.

Otoregresif LLM'ler: $P(x_t | x_{<t}) \rightarrow$ soldan sağa tek token

Difüzyon LLM'ler: $x_T \sim \text{Uniform}(V) \rightarrow$ iteratif gürültü giderme $\rightarrow x_0$

Bu modül, `model.forward()` çıktısından tuvali adım adım iyileştiren sampler ve yardımcı sınıfları içerir. HuggingFace `generation_diffusion_gemma.py` dosyasının öğrenme amaçlı sadeleştirilmiş halidir.



DiffusionGemmaGenerationConfig

`generate()` için difüzyon özel üretim parametreleri.

Klasik autoregressive GenerationConfig'e ek alanlar:

Denoising adımları (max_denoising_steps):

Tuval kaç kez refine edilecek. Her adımda model logits üretir, sampler bir kısmını kabul eder, geri kalanı yeniden gürültüler.

Sıcaklık programı (t_min, t_max):

Erken adımlarda yüksek sıcaklık → keşif; geç adımlarda düşük → kesinlik.
LinearTemperatureScheduleLogitsProcessor bunu uygular.

Sampler (sampler_config):

EntropyBoundSamplerConfig gibi bir yapı; entropi eşiği vb. taşır.

LinearTemperatureScheduleLogitsProcessor

Adıma bağlı lineer sıcaklık (temperature) programı.

Motivasyon

Difüzyonun başında tuval tamamen rastgeledir; model belirsizdir.

Yüksek sıcaklık, softmax dağılımını düzleştirerek çeşitli token örneklemeye izin verir. Son adımlarda tuval oturmuşken düşük sıcaklık, en olası tokenları seçmeye iter.

Formül

n. adımda ($0 \leq n \leq N$):

$$t(n) = t_{\min} + (t_{\max} - t_{\min}) \cdot (n / N)$$

Logits işlemcisi scores / $t(n)$ döndürür; bu, softmax öncesi ölçeklemedir.

Parametreler

`t_min` : float

Son denoising adımındaki sıcaklık (genelde düşük, örn. 0.1)

`t_max` : float

İlk adımdaki sıcaklık (genelde yüksek, örn. 1.0)

`max_denoising_steps` : int

Toplam adım sayısı `N`

EntropyBoundSampler

Entropi sınırına dayalı difüzyon sampler'ı.

Kaynak: <https://arxiv.org/pdf/2505.24857>

Neden entropi?

Her denoising adımında tüm tuvali bir anda değiştirmek, tokenlar arası güçlü bağımlılıklara yol açabilir. EntropyBoundSampler, her adımda yalnızca *birbirine yaklaşık bağımsız* (düşük birlikte entropi) tokenları kabul eder; kalan pozisyonlar sonraki adıma ertelenir.

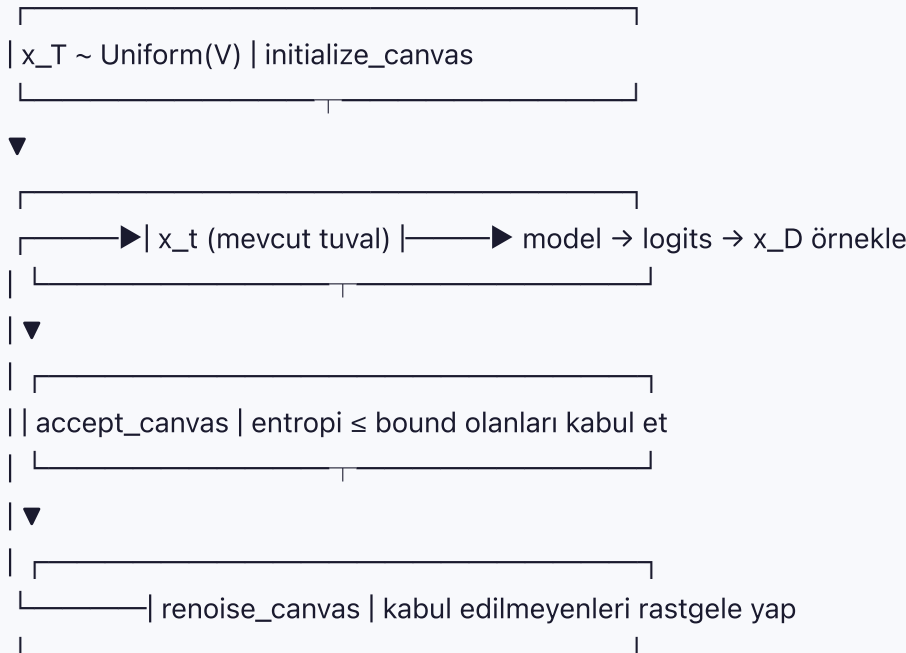
Kabul kriteri

Entropiye göre artan sırada tokenlar seçilir. k token kabul edilir ancak:

$$\sum_{i=1}^k H_i - \max(H_1, \dots, H_k) \leq \text{entropy_bound}$$

Sol taraf, k tokenın birlikte mutual information üst sınırıdır.

Algoritma döngüsü



EntropyBoundSampler.initialize_canvas()

Tuvali uniform rastgele tokenlarla doldurur (x_T).

Difüzyon sürecinin başlangıç noktası: tam gürültü, hiçbir pozisyon anlamlı değil. Model bu tuvali iteratif olarak anlamlı metne dönüştürür.

Returns

canvas_ids : LongTensor [batch_size, canvas_length]

```
147 |     def initialize_canvas(self, batch_size, device):
148 |         """
149 |         Tuvali uniform rastgele tokenlarla doldurur ( $x_T$ ).
150 |
151 |         Difüzyon sürecinin başlangıç noktası: tam gürültü, hiçbir pozisyon
152 |         anlamlı değil. Model bu tuvali iteratif olarak anlamlı metne dönüştürür.
153 |
154 |         Returns
155 |         -----
156 |         canvas_ids : LongTensor [batch_size, canvas_length]
157 |         """
158 |         canvas_ids = torch.randint(
159 |             low=0,
160 |             high=self.vocab_size,
161 |             size=(batch_size, self.canvas_length),
162 |             device=device,
163 |         )
164 |         return canvas_ids
165 |
166 |     def accept_canvas(
```

EntropyBoundSampler.accept_canvas()

Model tahminlerinden entropi sınırına göre token kabul eder.

Adımlar

1. Categorical(logits) ile her pozisyonun entropisi H hesaplanır
2. H 'ye göre artan sıralama (en emin tokenlar önce)
3. Kümülatif entropi – o adımdaki max entropi $\leq$ bound ise kabul
4. Kabul edilen pozisyonlar denoiser_canvas'tan, diğerleri current'tan

Parametreler

current_canvas : LongTensor [B, L]

Önceki adımın tuvali x_t

denoiser_canvas : LongTensor [B, L]

Modelden örneklenen aday tuval x_D

logits : FloatTensor [B, L, vocab]

Ham model çıkışı (softmax öncesi)

cur_step : int

Mevcut denoising adımı (ileride schedule için kullanılabilir)

Returns

accepted_canvas : LongTensor [B, L]

```
166 |     def accept_canvas(  
167 |         self,  
168 |         current_canvas,  
169 |         denoiser_canvas,  
170 |         logits,  
171 |         cur_step,  
172 |     ):  
173 |         """  
174 |         Model tahminlerinden entropi sınırına göre token kabul eder.  
175 |  
176 |         Adımlar  
177 |         -----  
178 |         1. Categorical(logits) ile her pozisyonun entropisi H hesaplanır  
179 |         2. H'ye göre artan sıralama (en emin tokenlar önce)  
180 |         3. Kümülatif entropi - o adımdaki max entropi ≤ bound ise kabul  
181 |         4. Kabul edilen pozisyonlar denoiser_canvas'tan, diğerleri current'tan  
182 |  
183 |         Parametreler  
184 |         -----  
185 |         current_canvas : LongTensor [B, L]
```

EntropyBoundSampler.renoise_canvas()

Kabul edilmeyen pozisyonları yeniden uniform rastgele tokenlarla doldurur.

Kabul edilen tokenlar sabit kalır (mask True). Renoise mask (~accepted) pozisyonları yeni rastgele ID alır; bir sonraki forward'da model bunları tekrar refine etmeye çalışır.

Parametreler

accepted_canvas : LongTensor [B, L]

cur_step : int — adım indeksi (gelecekte noise schedule için)

Returns

renoised_canvas : LongTensor [B, L]

```
215 | def renoise_canvas(self, accepted_canvas, cur_step):
216 |     """
217 |     Kabul edilmeyen pozisyonları yeniden uniform rastgele tokenlarla doldurur.
218 |
219 |     Kabul edilen tokenlar sabit kalır (mask True). Renoise mask (~accepted)
220 |     pozisyonları yeni rastgele ID alır; bir sonraki forward'da model bunları
221 |     tekrar refine etmeye çalışır.
222 |
223 |     Parametreler
224 |     -----
225 |     accepted_canvas : LongTensor [B, L]
226 |     cur_step : int – adım indeksi (gelecekte noise schedule için)
227 |
228 |     Returns
229 |     -----
230 |     renoised_canvas : LongTensor [B, L]
231 |     """
232 |     device = accepted_canvas.device
233 |     batch_size = accepted_canvas.shape[0]
234 |
```

5. modeling.py

DiffusionGemma — saf PyTorch model bileşenleri (öğrenme amaçlı).

Bu dosya HuggingFace `modeling\_diffusion\_gemma.py` dosyasının bağımlılıksız yeniden yazımıdır. Tüm sınıf ve fonksiyon docstring'leri Türkçedir; öğrencilerin mimariyi adım adım takip edebilmesi hedeflenmiştir.

DiffusionGemma nedir?

Otoregresif (AR) modeller token'ı token üretir; DiffusionGemma ise metni blok blok difüzyon ile üretir. Encoder prompt'u KV önbelleğine yazar; decoder tuvaldeki gürültülü tokenları iteratif olarak temizler.

Mimari özeti:

- Encoder: Gemma4 benzeri transformer + MoE; KV cache üretir
- Decoder: Aynı ağırlıkları paylaşır; encoder KV'yi salt okunur okur
- Self-conditioning: Önceki adım logits'leri decoder girdisine eklenir
- Blok difüzyon: Tuval parça parça gürültüden arındırılır

5.1 Temel yapı taşları

DiffusionGemmaTextxRotaryEmbedding

Rotary Positional Embedding (RoPE) katmanı.

DiffusionGemma'da farklı katman tipleri (full_attention, sliding_attention) farklı head boyutları ve frekans parametreleri kullanabilir. Bu sınıf her katman tipi için ayrı `inv\_freq` tamponu tutar.

RoPE fikri: pozisyon bilgisini vektöre eklemek yerine Q/K vektörlerini pozisyona göre karmaşık düzlemde döndürmek. Böylece attention, tokenlar arası göreceli mesafeyi doğal biçimde öğrenir.

```
25 | class DiffusionGemmaTextxRotaryEmbedding(nn.Module):
26 |     """
27 |         Rotary Positional Embedding (RoPE) katmanı.
28 |
29 |         DiffusionGemma'da farklı katman tipleri (full_attention, sliding_attention)
30 |         farklı head boyutları ve frekans parametreleri kullanabilir. Bu sınıf her
31 |         katman tipi için ayrı `inv_freq` tamponu tutar.
32 |
33 |         RoPE fikri: pozisyon bilgisini vektöre eklemek yerine Q/K vektörlerini
34 |         pozisyona göre karmaşık düzlemde döndürmek. Böylece attention, tokenlar
35 |         arası göreceli mesafeyi doğal biçimde öğrenir.
36 |         """
37 |     inv_freq: torch.Tensor
38 |
39 |     def __init__(self, config, device, layer_type):
40 |         super().__init__()
41 |         self.max_seq_len_cached = config.max_position_embeddings
42 |         self.original_max_seq_len = config.max_position_embeddings
43 |
44 |         self.config = config
45 |         # modeldeki benzersiz katman tipleri: örn. {"full_attention", "sliding_attention"}
46 |         self.layer_types = set(config.layer_types)
47 |         self.rope_init_fns = {}
48 |         self.rope_type = {}
49 |
50 |         for layer_type in self.layer_types:
51 |             rope_params = self.config.rope_parameters[layer_type]
52 |             if rope_params is None:
53 |                 continue
54 |
55 |             rope_init_fn = self.compute_default_rope_parameters
56 |             rope_type = rope_params.get("rope_type", "default")
```

DiffusionGemmaTextxRotaryEmbedding.forward()

Verilen pozisyonlar için cos/sin embedding üretir.

Args:

x: referans tensör (cihaz ve dtype için)

position_ids: [batch, seq_len] — her tokenın mutlak pozisyonu

layer_type: hangi katman tipinin frekanslarını kullanacağımız

Returns:

cos, sin: [batch, seq_len, head_dim] — Q/K'ya uygulanacak döndürme katsayıları

DiffusionGemmaRMSNorm

Root Mean Square Layer Normalization (RMSNorm).

LayerNorm'dan farkı: ortalamayı çıkarmaz, sadece L2 normuna göre ölçekler.

Gemma ailesinde standart normalizasyon yöntemidir.

Formül: $x / \sqrt{\text{mean}(x^2) + \text{eps}} * \text{weight}$

```

127 | class DiffusionGemmaRMSNorm(nn.Module):
128 |     """
129 |     Root Mean Square Layer Normalization (RMSNorm).
130 |
131 |     LayerNorm'dan farkı: ortalamayı çıkarmaz, sadece L2 normuna göre ölçekler.
132 |     Gemma ailesinde standart normalizasyon yöntemidir.
133 |
134 |     Formül:  $x / \sqrt{\text{mean}(x^2) + \text{eps}} * \text{weight}$ 
135 |     """
136 |     def __init__(self, dim, eps=1e-6, with_scale=True):
137 |         super().__init__()
138 |         self.eps = eps
139 |         self.with_scale = with_scale
140 |
141 |         if self.with_scale:
142 |             self.weight = nn.Parameter(torch.ones(dim), requires_grad=True)
143 |
144 |     def _norm(self, hidden_states):
145 |         mean_squared = hidden_states.pow(2).mean(-1, keepdim=True) + self.eps
146 |         return hidden_states * torch.pow(mean_squared, -0.5)
147 |
148 |     def forward(self, hidden_states):
149 |         """
150 |         RMSNorm uygular; isteğe bağlı öğrenilebilir weight ile ölçekler.
151 |
152 |         Args:
153 |             hidden_states: [..., dim]
154 |
155 |         Returns:
156 |             Normalize edilmiş tensör, girdi ile aynı şekil ve dtype
157 |         """
158 |         normed_output = self._norm(hidden_states.float())

```

DiffusionGemmaRMSNorm.forward()

RMSNorm uygular; isteğe bağlı öğrenilebilir weight ile ölçekler.

Args:

hidden_states: [..., dim]

Returns:

Normalize edilmiş tensör, girdi ile aynı şekil ve dtype

DiffusionGemmaClippableLiner

Giriş/çıkış değerlerini belirli aralıkta tutan doğrusal katman.

Eğitim stabilitesi için aktivasyonları clamp eder. DiffusionGemma'nın orijinal implementasyonunda `use\_clipped\_linears` bayrağı ile açılır.

```
164 | class DiffusionGemmaClippableLiner(nn.Module):
165 |     """
166 |     Giriş/çıkış değerlerini belirli aralıkta tutan doğrusal katman.
167 |
168 |     Eğitim stabilitesi için aktivasyonları clamp eder. DiffusionGemma'nın
169 |     orijinal implementasyonunda `use_clipped_linears` bayrağı ile açılır.
170 |     """
171 |     def __init__(
172 |         self,
173 |         config,
174 |         in_features,
175 |         out_features,
176 |     ) -> None:
177 |         super().__init__()
178 |         self.use_clipped_linears = config.use_clipped_linears
179 |         self.linear = nn.Linear(in_features, out_features, bias=False)
180 |
181 |         if self.use_clipped_linears:
182 |             self.register_buffer("input_min", torch.tensor(-float("inf")))
183 |             self.register_buffer("input_max", torch.tensor(float("inf")))
184 |             self.register_buffer("output_min", torch.tensor(-float("inf")))
185 |             self.register_buffer("output_max", torch.tensor(float("inf")))
186 |
187 |     def forward(self, hidden_states):
188 |         """
189 |         İsteğe bağlı giriş/çıkış clamp ile doğrusal dönüşüm uygular.
190 |
191 |         Args:
192 |             hidden_states: [..., in_features]
193 |
194 |         Returns:
195 |             [..., out_features]
```

DiffusionGemmaClippableLiner.forward()

İsteğe bağlı giriş/çıkış clamp ile doğrusal dönüşüm uygular.

Args:

hidden_states: [..., in_features]

Returns:

[..., out_features]

rotate_half

RoPE'nin temel döndürme adımı.

Vektörün ilk yarısı ile ikinci yarısını yer değiştirip işaret değiştirir.

Bu, karmaşık çarpım $(\cos + i \cdot \sin) \cdot (x_1 + i \cdot x_2)$ işleminin vektör karşılığıdır.

```
205 | def rotate_half(x):
206 |     """
207 |     RoPE'nin temel döndürme adımı.
208 |
209 |     Vektörün ilk yarısı ile ikinci yarısını yer değiştirip işaret değiştirir.
210 |     Bu, karmaşık çarpım  $(\cos + i \cdot \sin) \cdot (x_1 + i \cdot x_2)$  işleminin vektör karşılığıdır.
211 |     """
212 |     x1 = x[..., : x.shape[-1] // 2]
213 |     x2 = x[..., x.shape[-1] // 2 :]
214 |     return torch.cat((-x2, x1), dim=-1)
215 |
216 |
217 | def apply_rotary_pos_emb(x: torch.Tensor, cos: torch.Tensor, sin: torch.Tensor, unsqueeze_dim: int):
218 |     """
219 |     Q veya K tensörüne RoPE uygular.
220 |
221 |     Formül:  $x_{rotated} = x * \cos + rotate\_half(x) * \sin$ 
222 |
223 |     unsqueeze_dim: cos/sin'in Q/K ile broadcast edilebilmesi için hangi eksene
224 |     boyut ekleneceğini belirler. Attention'da Q/K şekli [batch, heads, seq, dim]
225 |     olduğundan genelde unsqueeze_dim=2 kullanılır.
226 |     """
227 |     cos = cos.unsqueeze(unsqueeze_dim)
228 |     sin = sin.unsqueeze(unsqueeze_dim)
229 |     return (x * cos) + (rotate_half(x) * sin)
230 |
231 |
232 | def repeat_kv(hidden_states, n_rep):
233 |     """
234 |     Grouped Query Attention (GQA) için K/V head'lerini çoğaltır.
235 |
236 |     Az sayıda K/V head'i (num_key_value_heads) vardır; Q head sayısı (num_attention_heads)
```

apply_rotary_pos_emb

Q veya K tensörüne RoPE uygular.

Formül: $x_{\text{rotated}} = x * \cos + \text{rotate_half}(x) * \sin$

unsqueeze_dim: cos/sin'in Q/K ile broadcast edilebilmesi için hangi eksene boyut ekleneceğini belirler. Attention'da Q/K şekli [batch, heads, seq, dim] olduğundan genelde unsqueeze_dim=2 kullanılır.

```

217 | def apply_rotary_pos_emb(x: torch.Tensor, cos: torch.Tensor, sin: torch.Tensor, unsqueeze
218 |     """
219 |     Q veya K tensörüne RoPE uygular.
220 |
221 |     Formül:  $x_{\text{rotated}} = x * \cos + \text{rotate\_half}(x) * \sin$ 
222 |
223 |     unsqueeze_dim: cos/sin'in Q/K ile broadcast edilebilmesi için hangi eksene
224 |     boyut ekleneceğini belirler. Attention'da Q/K şekli [batch, heads, seq, dim]
225 |     olduğundan genelde unsqueeze_dim=2 kullanılır.
226 |     """
227 |     cos = cos.unsqueeze(unsqueeze_dim)
228 |     sin = sin.unsqueeze(unsqueeze_dim)
229 |     return (x * cos) + (rotate_half(x) * sin)
230 |
231 |
232 | def repeat_kv(hidden_states, n_rep):
233 |     """
234 |     Grouped Query Attention (GQA) için K/V head'lerini çoğaltır.
235 |
236 |     Az sayıda K/V head'i (num_key_value_heads) vardır; Q head sayısı (num_attention_heads
237 |     daha fazladır. Her K/V head'i, ona karşılık gelen Q head grubu kadar tekrarlanır.
238 |
239 |     torch.repeat_interleave(x, dim=1, repeats=n_rep) ile aynı işi yapar,
240 |     ama expand+reshape ile daha verimli.
241 |
242 |     Girdi : [batch, num_kv_heads, seq_len, head_dim]
243 |     Çıktı : [batch, num_attention_heads, seq_len, head_dim]
244 |     """
245 |     if n_rep == 1:
246 |         return hidden_states
247 |     b, nkvh, sl, hd = hidden_states.shape
248 |     hidden_states = hidden_states[:, :, None, :, :].expand(b, nkvh, n_rep, sl, hd)

```

repeat_kv

Grouped Query Attention (GQA) için K/V head'lerini çoğaltır.

Az sayıda K/V head'i (num_key_value_heads) vardır; Q head sayısı (num_attention_heads) daha fazladır. Her K/V head'i, ona karşılık gelen Q head grubu kadar tekrarlanır.

torch.repeat_interleave(x, dim=1, repeats=n_rep) ile aynı işi yapar, ama expand+reshape ile daha verimli.

Girdi : [batch, num_kv_heads, seq_len, head_dim]

Çıktı : [batch, num_attention_heads, seq_len, head_dim]

```

232 | def repeat_kv(hidden_states, n_rep):
233 |     """
234 |     Grouped Query Attention (GQA) için K/V head'lerini çoğaltır.
235 |
236 |     Az sayıda K/V head'i (num_key_value_heads) vardır; Q head sayısı (num_attention_heads)
237 |     daha fazladır. Her K/V head'i, ona karşılık gelen Q head grubu kadar tekrarlanır.
238 |
239 |     torch.repeat_interleave(x, dim=1, repeats=n_rep) ile aynı işi yapar,
240 |     ama expand+reshape ile daha verimli.
241 |
242 |     Girdi : [batch, num_kv_heads, seq_len, head_dim]
243 |     Çıktı : [batch, num_attention_heads, seq_len, head_dim]
244 |     """
245 |     if n_rep == 1:
246 |         return hidden_states
247 |     b, nkvh, sl, hd = hidden_states.shape
248 |     hidden_states = hidden_states[:, :, None, :, :].expand(b, nkvh, n_rep, sl, hd)
249 |     return hidden_states.reshape(b, nkvh * n_rep, sl, hd)
250 |
251 |
252 | def eager_attention_forward(
253 |     module: nn.Module,
254 |     query,
255 |     key,
256 |     value,
257 |     attention_mask,
258 |     dropout=0.0,
259 |     scaling=None,
260 |     softcap=None,
261 |     **kwargs
262 | ):
263 |     """

```

eager_attention_forward

Standart scaled dot-product attention (saf PyTorch, Flash/SDPA yok).

Adımlar:

1. K/V head'lerini GQA için tekrarla
2. $\text{attn_scores} = Q @ K^T * \text{scaling}$
3. (opsiyonel) softcap ile skorları sınırla
4. attention_mask ekle (causal/sliding mask burada uygulanır)
5. softmax $\rightarrow$ dropout $\rightarrow @ V$

DiffusionGemma'da scaling=1.0 kullanılır (klasik $1/\sqrt{d}$ değil).

Returns:

attn_output: [batch, seq_len, num_heads, head_dim]

attn_weights: [batch, num_heads, seq_len, seq_len]

```

252 | def eager_attention_forward(
253 |     module: nn.Module,
254 |     query,
255 |     key,
256 |     value,
257 |     attention_mask,
258 |     dropout=0.0,
259 |     scaling=None,
260 |     softcap=None,
261 |     **kwargs
262 | ):
263 |     """
264 |     Standart scaled dot-product attention (saf PyTorch, Flash/SDPA yok).
265 |
266 |     Adımlar:
267 |         1. K/V head'lerini GQA için tekrarla
268 |         2.  $\text{attn\_scores} = Q @ K^T * \text{scaling}$ 
269 |         3. (opsiyonel) softcap ile skorları sınırla
270 |         4. attention_mask ekle (causal/sliding mask burada uygulanır)
271 |         5. softmax  $\rightarrow$  dropout  $\rightarrow @ V$ 
272 |
273 |     DiffusionGemma'da scaling=1.0 kullanılır (klasik  $1/\sqrt{d}$  değil).
274 |
275 |     Returns:
276 |         attn_output: [batch, seq_len, num_heads, head_dim]
277 |         attn_weights: [batch, num_heads, seq_len, seq_len]
278 |     """
279 |     if scaling is None:
280 |         scaling = module.head_dim**-0.5
281 |
282 |     key_states = repeat_kv(key, module.num_key_value_groups)
283 |     value_states = repeat_kv(value, module.num_key_value_groups)

```


5.2 Attention katmanları



DiffusionGemmaEncoderTextAttention

DiffusionGemma encoder (denoiser) attention katmanı.

Gemma4 attention'a benzer; farklar:

- Paylaşımlı KV cache mantığı yok (difüzyon modelinde kullanılmıyor)
- `full\_attention` katmanlarında V projeksiyonu yok; $V = K$ (weight tying)
- `sliding\_attention` katmanlarında ayrı V projeksiyonu var
- Global (full) katmanlarda head_dim=512, sliding'de head_dim=256

is_causal:

- use_bidirectional_attention="all" ise False → tüm tokenlar birbirini görür
- aksi halde True → causal (geleceği görme) mask uygulanır

```

307 | class DiffusionGemmaEncoderTextAttention(nn.Module):
308 |     """
309 |     DiffusionGemma encoder (denoiser) attention katmanı.
310 |
311 |     Gemma4 attention'a benzer; farklar:
312 |     - Paylaşımlı KV cache mantığı yok (difüzyon modelinde kullanılmıyor)
313 |     - `full_attention` katmanlarında V projeksiyonu yok;  $V = K$  (weight tying)
314 |     - `sliding_attention` katmanlarında ayrı V projeksiyonu var
315 |     - Global (full) katmanlarda head_dim=512, sliding'de head_dim=256
316 |
317 |     is_causal:
318 |     - use_bidirectional_attention="all" ise False → tüm tokenlar birbirini görür
319 |     - aksi halde True → causal (geleceği görme) mask uygulanır
320 |     """
321 |     def __init__(self, config, layer_idx):
322 |         super().__init__()
323 |         self.is_causal = config.use_bidirectional_attention != 'all'
324 |
325 |         self.layer_type = config.layer_types[layer_idx] if hasattr(config, "layer_types")
326 |         self.config = config
327 |         self.layer_idx = layer_idx
328 |         self.is_sliding = self.layer_type == "sliding_attention"
329 |         self.sliding_window = config.sliding_window if self.is_sliding else None
330 |
331 |         # full attention katmanları daha büyük head kullanır (global context)
332 |         self.head_dim = config.global_head_dim if not self.is_sliding and config.global_h
333 |         num_key_value_heads = config.num_global_key_value_heads if not self.is_sliding el
334 |         self.num_key_value_groups = config.num_attention_heads // num_key_value_heads
335 |         self.scaling = 1.0 # Gemma scaling; sqrt(head_dim) ile bölünmez
336 |         self.attention_dropout = self.config.attention_dropout
337 |
338 |         self.q_proj = nn.Linear(

```

DiffusionGemmaEncoderTextAttention.forward()

Args:

hidden_states: [batch, seq_len, hidden_size]

position_embeddings: (cos, sin) tuple — RoPE katsayıları

attention_mask: additive mask veya None

past_key_values: önceki adımlardan KV cache (encoder tarafında)

Returns:

attn_output: [batch, seq_len, hidden_size]

attn_weights: attention ağırlıkları (eager modda)

DiffusionGemmaDecoderTextAttention

DiffusionGemma decoder attention katmanı — tuval self-attention + encoder çapraz dikkat.

DiffusionGemma mimarisinde encoder, kullanıcı prompt'unu işleyerek her katmanda Key/Value (K/V) tensörlerini bir önbelleğe (KV cache) yazar. Decoder ise difüzyon tuvalindeki (canvas) gürültülü tokenları adım adım temizlerken bu önbelleği *salt okunur* olarak kullanır; decoder kendi K/V'sini önbelleğe eklemeyebilir.

Bu katmanın görevi, tuval tokenlarından üretilen Query'lerin hem tuvalin kendi K/V'sine hem de encoder'ın önbelleğindeki K/V'ye bakmasını sağlamaktır. Pratikte attention şu şekilde çalışır:

1. Tuval gizli durumlarından Q, K, V projeksiyonları hesaplanır (RoPE uygulanır).
2. Encoder önbelleğindeki K/V, mevcut tuval K/V ile seq ekseninde birleştirilir (torch.cat). past_key_values.update() *çağrılmaz* — önbellek değişmez.
3. Birleşik K/V üzerinde iki yönlü (bidirectional) scaled dot-product attention uygulanır (is_causal=False).

Encoder attention'dan üç kritik fark:

- Paylaşımlı KV katmanı mantığı yok (PLE kullanılmıyor).
- KV önbelleği güncellenmez; decoder encoder çıktısını sadece okur.
- is_causal her zaman False; config.use_bidirectional_attention yalnızca encoder'ı etkiler, decoder attention her zaman çift yönlüdür.

Öğrenci notu: Klasik encoder-decoder modellerde decoder, encoder'ın son gizli durumlarını cross-attention ile okur. DiffusionGemma'da ise decoder doğrudan encoder'ın *tüm ara katman K/V önbelleğini* okur; bu sayede tuval tokenları prompt bağlamına her katmanda erişebilir.

```

426 | class DiffusionGemmaDecoderTextAttention(nn.Module):
427 |     """
428 |     DiffusionGemma decoder attention katmanı – tuval self-attention + encoder çapraz dikk
429 |
430 |     DiffusionGemma mimarisinde encoder, kullanıcı prompt'unu işleyerek her katmanda
431 |     Key/Value (K/V) tensörlerini bir önbelleğe (KV cache) yazar. Decoder ise difüzyon
432 |     tuvalindeki (canvas) gürültülü tokenları adım adım temizlerken bu önbelleği
433 |     *salt okunur* olarak kullanır; decoder kendi K/V'sini önbelleğe eklemeyiz.
434 |
435 |     Bu katmanın görevi, tuval tokenlarından üretilen Query'lerin hem tuvalin kendi
436 |     K/V'sine hem de encoder'ın önbelleğindeki K/V'ye bakmasını sağlamaktır. Pratikte
437 |     attention şu şekilde çalışır:
438 |
439 |     1. Tuval gizli durumlarından Q, K, V projeksiyonları hesaplanır (RoPE uygulanır).
440 |     2. Encoder önbelleğindeki K/V, mevcut tuval K/V ile seq ekseninde birleştirilir
441 |        (torch.cat). past_key_values.update() *çağrılmaz* – önbellek değişmez.
442 |     3. Birleşik K/V üzerinde iki yönlü (bidirectional) scaled dot-product attention
443 |        uygulanır (is_causal=False).
444 |
445 |     Encoder attention'dan üç kritik fark:
446 |     - Paylaşımlı KV katmanı mantığı yok (PLE kullanılmıyor).
447 |     - KV önbelleği güncellenmez; decoder encoder çıktısını sadece okur.
448 |     - is_causal her zaman False; config.use_bidirectional_attention yalnızca encoder'ı
449 |       etkiler, decoder attention her zaman çift yönlüdür.
450 |
451 |     Öğrenci notu: Klasik encoder-decoder modellerde decoder, encoder'ın son gizli
452 |     durumlarını cross-attention ile okur. DiffusionGemma'da ise decoder doğrudan
453 |     encoder'ın *tüm ara katman K/V önbelleğini* okur; bu sayede tuval tokenları
454 |     prompt bağlamına her katmanda erişebilir.
455 |     """
456 |     def __init__(self, config, layer_idx):
457 |         super().__init__()

```

DiffusionGemmaDecoderTextAttention.forward()

Tuval tokenları için çapraz dikkat + self-attention hesaplar.

Encoder KV önbelleği salt okunur: K/V birleştirilir, önbellek güncellenmez.

Args:

hidden_states: [batch, canvas_len, hidden_size] — tuval gizli durumları

position_embeddings: (cos, sin) — tuval pozisyonları için RoPE katsayıları

attention_mask: encoder önbelleği + tuval için additive mask

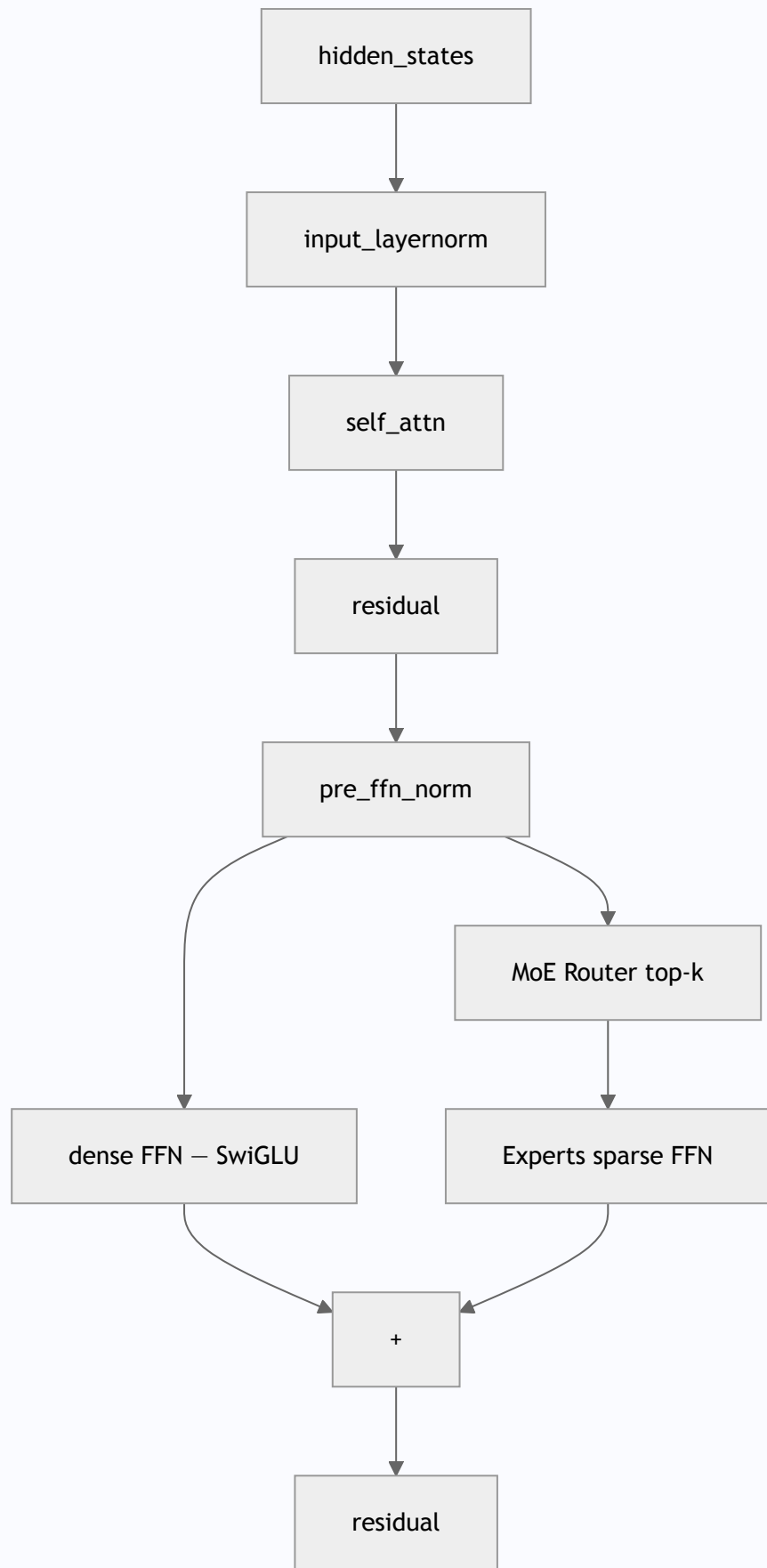
past_key_values: encoder'ın yazdığı salt okunur KV önbelleği

Returns:

attn_output: [batch, canvas_len, hidden_size]

attn_weights: attention ağırlıkları (eager modda)

5.3 FFN ve MoE



DiffusionGemmaText4MLP

Gemma4 tarzı kapılı feed-forward ağı (SwiGLU benzeri).

Her token bağımsız işlenir. gate_proj ve up_proj paralel çalışır; aktivasyon gate üzerinden uygulanır, up ile çarpılır ve down_proj ile gizli boyuta geri projekte edilir.

Formül: $\text{down_proj}(\text{act_fn}(\text{gate_proj}(x)) * \text{up_proj}(x))$

```

563 | class DiffusionGemmaText4MLP(nn.Module):
564 |     """
565 |         Gemma4 tarzı kapılı feed-forward ağı (SwiGLU benzeri).
566 |
567 |         Her token bağımsız işlenir. gate_proj ve up_proj paralel çalışır; aktivasyon
568 |         gate üzerinden uygulanır, up ile çarpılır ve down_proj ile gizli boyuta geri
569 |         projekte edilir.
570 |
571 |         Formül: down_proj( act_fn(gate_proj(x)) * up_proj(x) )
572 |         """
573 |     def __init__(self, config, layer_idx):
574 |         super().__init__()
575 |         self.config = config
576 |         self.hidden_size = config.hidden_size
577 |         self.intermediate_size = config.intermediate_size
578 |
579 |         self.gate_proj = nn.Linear(self.hidden_size, self.intermediate_size, bias=False)
580 |         self.up_proj = nn.Linear(self.hidden_size, self.intermediate_size, bias=False)
581 |         self.down_proj = nn.Linear(self.intermediate_size, self.hidden_size, bias=False)
582 |         self.act_fn = config.hidden_activation
583 |
584 |     def forward(self, x):
585 |         """
586 |         Kapılı MLP ileri geçişi.
587 |
588 |         Args:
589 |             x: [batch, seq_len, hidden_size]
590 |
591 |         Returns:
592 |             [batch, seq_len, hidden_size]
593 |         """
594 |         up = self.up_proj(x)

```

DiffusionGemmaText4MLP.forward()

Kapılı MLP ileri geçişi.

Args:

x: [batch, seq_len, hidden_size]

Returns:

[batch, seq_len, hidden_size]

DiffusionGemmaTextRouter

Mixture-of-Experts (MoE) yönlendirici (router) katmanı.

Her token için hangi uzmanların (expert) devreye gireceğine karar verir.

DiffusionGemma'da encoder ve decoder katmanları hem sabit MLP hem de MoE yolunu paralel çalıştırır; router yalnızca MoE kolundaki token yönlendirmesinden sorumludur.

Çalışma adımları:

1. Girdi RMSNorm ile normalize edilir, $\text{hidden_size}^{-0.5}$ ve öğrenilebilir scale ile ölçeklenir.
2. Linear projeksiyon tüm uzmanlara ham skor üretir.
3. Softmax (fp32) ile olasılık dağılımı hesaplanır.
4. `top_k_experts` kadar uzman seçilir; ağırlıklar token başına 1'e normalize edilir.
5. `per_expert_scale` ile uzman bazlı ek ölçekleme uygulanır.

Çıktılar:

- `router_probabilities`: tüm uzman olasılıkları (eğitim/izleme için)
- `top_k_weights`: seçilen uzmanların normalize ağırlıkları
- `top_k_index`: seçilen uzman indeksleri

```

601 | class DiffusionGemmaTextRouter(nn.Module):
602 |     """
603 |     Mixture-of-Experts (MoE) yönlendirici (router) katmanı.
604 |
605 |     Her token için hangi uzmanların (expert) devreye gireceğine karar verir.
606 |     DiffusionGemma'da encoder ve decoder katmanları hem sabit MLP hem de MoE
607 |     yolunu paralel çalıştırır; router yalnızca MoE kolundaki token yönlendirmesinden
608 |     sorumludur.
609 |
610 |     Çalışma adımları:
611 |     1. Girdi RMSNorm ile normalize edilir,  $\text{hidden\_size}^{-0.5}$  ve öğrenilebilir scale
612 |        ile ölçeklenir.
613 |     2. Linear projeksiyon tüm uzmanlara ham skor üretir.
614 |     3. Softmax (fp32) ile olasılık dağılımı hesaplanır.
615 |     4. top_k_experts kadar uzman seçilir; ağırlıklar token başına 1'e normalize edilir.
616 |     5. per_expert_scale ile uzman bazlı ek ölçekleme uygulanır.
617 |
618 |     Çıktılar:
619 |     - router_probabilities: tüm uzman olasılıkları (eğitim/izleme için)
620 |     - top_k_weights: seçilen uzmanların normalize ağırlıkları
621 |     - top_k_index: seçilen uzman indeksleri
622 |     """
623 |     def __init__(self, config: DiffusionGemmaTextConfig):
624 |         super().__init__()
625 |         self.config = config
626 |         self.hidden_size = config.hidden_size
627 |         self.scalar_root_size = self.hidden_size**-0.5
628 |         self.eps = config.rms_norm_eps
629 |
630 |         self.norm = DiffusionGemmaRMSNorm(self.hidden_size, eps=self.eps, with_scale=False)
631 |         self.proj = nn.Linear(config.hidden_size, config.num_experts, bias=False)
632 |         self.scale = nn.Parameter(torch.ones(self.hidden_size))

```

DiffusionGemmaTextRouter.forward()

Token başına uzman seçimi ve ağırlıklandırma yapar.

Args:

`hidden_states`: [batch*seq, hidden_size] veya [batch, seq, hidden_size]

düzleştirilmiş token temsilleri

Returns:

`router_probabilities`: [..., num_experts] — tüm uzman softmax olasılıkları

`top_k_weights`: [..., top_k] — seçilen uzmanların normalize ağırlıkları

`top_k_index`: [..., top_k] — seçilen uzman indeksleri

DiffusionGemmaTextExperts

MoE uzman ağırlıklarının 3D tensör olarak saklandığı koleksiyon.

Her uzman, bağımsız bir kapılı FFN'dir. Ağırlıklar `nn.Parameter` olarak `[num_experts, ...]` şeklinde tutulur; bu sayede tüm uzmanlar tek modülde vektörize edilebilir.

İleri geçişte yalnızca router'ın seçtiği uzmanlar çalıştırılır (seyrek aktivasyon).

Her token, `top_k` uzmanın çıktısının ağırlıklı toplamını alır.

```

669 | class DiffusionGemmaTextExperts(nn.Module):
670 |     """
671 |     MoE uzman ağırlıklarının 3D tensör olarak saklandığı koleksiyon.
672 |
673 |     Her uzman, bağımsız bir kapılı FFN'dir. Ağırlıklar nn.Parameter olarak
674 |     [num_experts, ...] şeklinde tutulur; bu sayede tüm uzmanlar tek modülde
675 |     vektörize edilebilir.
676 |
677 |     İleri geçişte yalnızca router'ın seçtiği uzmanlar çalıştırılır (seyrek aktivasyon).
678 |     Her token, top_k uzmanın çıktısının ağırlıklı toplamını alır.
679 |     """
680 |     def __init__(self, config: DiffusionGemmaTextConfig):
681 |         super().__init__()
682 |         self.num_experts = config.num_experts
683 |         self.hidden_dim = config.hidden_size
684 |         self.intermediate_dim = config.moe_intermediate_size
685 |         self.gate_up_proj = nn.Parameter(torch.empty(self.num_experts, 2 * self.intermedi
686 |         self.down_proj = nn.Parameter(torch.empty(self.num_experts, self.hidden_dim, self
687 |         self.act_fn = ACT2FN[config.hidden_activation]
688 |
689 |     def forward(
690 |         self,
691 |         hidden_states: torch.Tensor,
692 |         top_k_index: torch.Tensor,
693 |         top_k_weights: torch.Tensor,
694 |     ) -> torch.Tensor:
695 |         """
696 |         Seçilen uzmanları token bazında çalıştırır ve ağırlıklı toplam döner.
697 |
698 |         Yalnızca en az bir token tarafından seçilen uzmanlar döngüye alınır;
699 |         bu seyrek yürütme MoE'nin hesaplama maliyetini düşürür.
700 |

```

DiffusionGemmaTextExperts.forward()

Seçilen uzmanları token bazında çalıştırır ve ağırlıklı toplam döner.

Yalnızca en az bir token tarafından seçilen uzmanlar döngüye alınır; bu seyrek yürütme MoE'nin hesaplama maliyetini düşürür.

Args:

hidden_states: [num_tokens, hidden_dim] — düzleştirilmiş token girdileri

top_k_index: [num_tokens, top_k] — her token için seçilen uzman indeksleri

top_k_weights: [num_tokens, top_k] — uzman ağırlıkları

Returns:

[num_tokens, hidden_dim] — uzman çıktılarının ağırlıklı toplamı

5.4 Transformer katmanları

DiffusionGemmaEncoderTextLayer

DiffusionGemma encoder (denoiser bağlam encoder'ı) transformer katmanı.

Standart transformer bloğuna ek olarak *çift feed-forward* yapısı vardır:
sabit MLP ve MoE kolu paralel çalışır, çıktıları toplanır.

Katman akışı (pre-norm + residual):

1. input_layernorm → self-attention → post_attention_layernorm → residual
2. pre_feedforward_layernorm → MLP → post_feedforward_layernorm_1 (MLP kolu)
3. Paralel MoE kolu: residual (MLP öncesi) → router → experts → post_norm_2
4. hidden = MLP_kolu + MoE_kolu → post_feedforward_layernorm → residual
5. layer_scalar ile çıkış ölçeklenir

Gemma4TextDecoderLayer'dan farklar:

- PLE (paylaşımlı KV) kod yolu yok
- shared_kv_states taşınmıyor
- EncoderTextAttention kullanır (KV ön belleği güncellenir)

```

730 | class DiffusionGemmaEncoderTextLayer(GradientCheckpointingLayer):
731 |     """
732 |     DiffusionGemma encoder (denoiser bağlam encoder'ı) transformer katmanı.
733 |
734 |     Standart transformer bloğuna ek olarak *çift feed-forward* yapısı vardır:
735 |     sabit MLP ve MoE kolu paralel çalışır, çıktıları toplanır.
736 |
737 |     Katman akışı (pre-norm + residual):
738 |     1. input_layernorm → self-attention → post_attention_layernorm → residual
739 |     2. pre_feedforward_layernorm → MLP → post_feedforward_layernorm_1 (MLP kolu)
740 |     3. Paralel MoE kolu: residual (MLP öncesi) → router → experts → post_norm_2
741 |     4. hidden = MLP_kolu + MoE_kolu → post_feedforward_layernorm → residual
742 |     5. layer_scalar ile çıkış ölçeklenir
743 |
744 |     Gemma4TextDecoderLayer'dan farklar:
745 |     - PLE (paylaşımlı KV) kod yolu yok
746 |     - shared_kv_states taşınmıyor
747 |     - EncoderTextAttention kullanır (KV önbelleği güncellenir)
748 |     """
749 |     def __init__(self, config: DiffusionGemmaConfig, layer_idx: int):
750 |         super().__init__()
751 |         self.config = config
752 |         self.hidden_size = config.hidden_size
753 |         self.layer_idx = layer_idx
754 |         self.self_attn = DiffusionGemmaEncoderTextAttention(config=config, layer_idx=layer_idx)
755 |         self.mlp = DiffusionGemmaText4MLP(config, layer_idx)
756 |         self.input_layernorm = DiffusionGemmaRMSNorm(self.hidden_size, eps=config.rms_norm_eps)
757 |         self.post_attention_layernorm = DiffusionGemmaRMSNorm(self.hidden_size, eps=config.rms_norm_eps)
758 |         self.pre_feedforward_layernorm = DiffusionGemmaRMSNorm(self.hidden_size, eps=config.rms_norm_eps)
759 |         self.post_feedforward_layernorm = DiffusionGemmaRMSNorm(self.hidden_size, eps=config.rms_norm_eps)
760 |         self.register_buffer("layer_scalar", torch.ones(1))
761 |

```

DiffusionGemmaEncoderTextLayer.forward()

Encoder katmanının tam ileri geçişi: attention + çift FFN (MLP + MoE).

Args:

hidden_states: [batch, seq_len, hidden_size]

position_embeddings: (cos, sin) RoPE katsayıları

attention_mask: additive attention mask

position_ids: token pozisyon kimlikleri

past_key_values: KV önbelleği (encoder tarafında güncellenir)

Returns:

[batch, seq_len, hidden_size]

DiffusionGemmaDecoderTextLayer

DiffusionGemma decoder transformer katmanı.

Encoder katmanıyla aynı çift FFN (MLP + MoE) yapısına sahiptir; tek kritik fark attention modülüdür: DiffusionGemmaDecoderTextAttention kullanır.

Decoder attention özellikleri:

- Encoder KV önbelleğini salt okunur okur (çapraz dikkat)
- Tuval tokenları arasında iki yönlü self-attention
- past_key_values güncellenmez

Gemma4TextDecoderLayer'dan farklar:

- PLE kod yolu yok
- shared_kv_states taşınmıyor
- DecoderTextAttention ile encoder önbelleğine salt okunur erişim

```

828 | class DiffusionGemmaDecoderTextLayer(GradientCheckpointingLayer):
829 |     """
830 |         DiffusionGemma decoder transformer katmanı.
831 |
832 |         Encoder katmanıyla aynı çift FFN (MLP + MoE) yapısına sahiptir; tek kritik
833 |         fark attention modülüdür: DiffusionGemmaDecoderTextAttention kullanır.
834 |
835 |         Decoder attention özellikleri:
836 |         - Encoder KV önbelleğini salt okunur okur (çapraz dikkat)
837 |         - Tuval tokenları arasında iki yönlü self-attention
838 |         - past_key_values güncellenmez
839 |
840 |         Gemma4TextDecoderLayer'dan farklar:
841 |         - PLE kod yolu yok
842 |         - shared_kv_states taşınmıyor
843 |         - DecoderTextAttention ile encoder önbelleğine salt okunur erişim
844 |     """
845 |     def __init__(self, config: DiffusionGemmaConfig, layer_idx: int):
846 |         super().__init__()
847 |         self.config = config
848 |         self.hidden_size = config.hidden_size
849 |         self.layer_idx = layer_idx
850 |         self.self_attn = DiffusionGemmaDecoderTextAttention(config=config, layer_idx=layer_idx)
851 |         self.mlp = DiffusionGemmaText4MLP(config, layer_idx)
852 |         self.input_layernorm = DiffusionGemmaRMSNorm(self.hidden_size, eps=config.rms_norm_eps)
853 |         self.post_attention_layernorm = DiffusionGemmaRMSNorm(self.hidden_size, eps=config.rms_norm_eps)
854 |         self.pre_feedforward_layernorm = DiffusionGemmaRMSNorm(self.hidden_size, eps=config.rms_norm_eps)
855 |         self.post_feedforward_layernorm = DiffusionGemmaRMSNorm(self.hidden_size, eps=config.rms_norm_eps)
856 |         self.register_buffer("layer_scalar", torch.ones(1))
857 |
858 |         self.router = DiffusionGemmaTextRouter(config)
859 |         self.experts = DiffusionGemmaTextExperts(config)

```

DiffusionGemmaDecoderTextLayer.forward()

Decoder katmanının tam ileri geiři: apraz dikkat + ift FFN (MLP + MoE).

Args:

hidden_states: [batch, canvas_len, hidden_size]

position_embeddings: (cos, sin) RoPE katsayıları

attention_mask: encoder nbelleęi + tuval iin mask

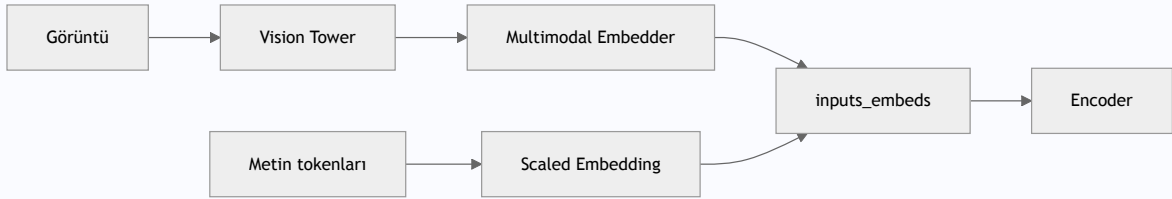
position_ids: tuval token pozisyon kimlikleri

past_key_values: encoder'ın salt okunur KV nbelleęi

Returns:

[batch, canvas_len, hidden_size]

5.5 Gmme ve ok modlu



DiffusionGemmaTextScaledWordEmbedding

Öğrenilen gömme vektörlerini $\sqrt{\text{hidden_size}}$ ile ölçekleyen word embedding.

Gemma ailesinde embedding çıkışı $\text{hidden_size}^{0.5}$ ile çarpılır; bu sayede gömme büyüklüğü transformer katmanlarıyla uyumlu kalır. `nn.Embedding.forward` sonucuna `embed_scale` tamponu uygulanır.

```

924 | class DiffusionGemmaTextScaledWordEmbedding(nn.Embedding):
925 |     """
926 |     Öğrenilen gömme vektörlerini sqrt(hidden_size) ile ölçekleyen word embedding.
927 |
928 |     Gemma ailesinde embedding çıkışı hidden_size^0.5 ile çarpılır; bu sayede
929 |     gömme büyüklüğü transformer katmanlarıyla uyumlu kalır. nn.Embedding.forward
930 |     sonucuna embed_scale tamponu uygulanır.
931 |     """
932 |     def __init__(self, num_embeddings: int, embedding_dim: int, padding_idx: int, embed_s
933 |                  super().__init__(num_embeddings, embedding_dim, padding_idx)
934 |                  self.scalar_embed_scale = embed_scale
935 |                  self.register_buffer("embed_scale", torch.tensor(embed_scale), persistent=False)
936 |
937 |     def forward(self, input_ids: torch.Tensor):
938 |         """
939 |         Token ID'lerini ölçeklenmiş gömme vektörlerine dönüştürür.
940 |
941 |         Args:
942 |             input_ids: [batch, seq_len] — token kimlikleri
943 |
944 |         Returns:
945 |             [batch, seq_len, embedding_dim]
946 |         """
947 |         return super().forward(input_ids) * self.embed_scale.to(self.weight.dtype)
948 |
949 |
950 | class DiffusionGemmaMultimodalEmbedder(nn.Module):
951 |     """
952 |     Çok modlu (görüntü vb.) özellikleri dil modeli uzayına projekte eden katman.
953 |
954 |     Vision tower çıktıları, önce RMSNorm ile normalize edilir, ardından Linear
955 |     katmanı ile text hidden_size boyutuna projekte edilir. Böylece görüntü patch

```

DiffusionGemmaTextScaledWordEmbedding.forward()

Token ID'lerini ölçeklenmiş gömme vektörlerine dönüştürür.

Args:

`input_ids`: [batch, seq_len] — token kimlikleri

Returns:

[batch, seq_len, embedding_dim]

DiffusionGemmaMultimodalEmbedder

Çok modlu (görüntü vb.) özellikleri dil modeli uzayına projekte eden katman.

Vision tower çıktıları, önce RMSNorm ile normalize edilir, ardından Linear katmanı ile text hidden_size boyutuna projekte edilir. Böylece görüntü patch temsilleri, metin token gömmeleriyle aynı uzayda birleştirilebilir.

```

950 | class DiffusionGemmaMultimodalEmbedder(nn.Module):
951 |     """
952 |     Çok modlu (görüntü vb.) özellikleri dil modeli uzayına projekte eden katman.
953 |
954 |     Vision tower çıktıları, önce RMSNorm ile normalize edilir, ardından Linear
955 |     katmanı ile text hidden_size boyutuna projekte edilir. Böylece görüntü patch
956 |     temsilleri, metin token gömmeleriyle aynı uzayda birleştirilebilir.
957 |     """
958 |     def __init__(
959 |         self,
960 |         multimodal_config: PreTrainedConfig,
961 |         text_config: DiffusionGemmaTextConfig,
962 |     ):
963 |         super().__init__()
964 |
965 |         self.multimodal_hidden_size = getattr(multimodal_config, "output_proj_dims", mult
966 |         self.eps = multimodal_config.rms_norm_eps
967 |         self.text_hidden_size = text_config.hidden_size
968 |         self.embedding_projection = nn.Linear(self.multimodal_hidden_size, self.text_hidd
969 |         self.embedding_pre_projection_norm = DiffusionGemmaRMSNorm(
970 |             self.multimodal_hidden_size, eps=self.eps, with_scale=False
971 |         )
972 |
973 |     def forward(self, inputs_embeds: torch.Tensor) -> torch.Tensor:
974 |         """
975 |         Çok modlu soft token gömmelerini dil modeli boyutuna projekte eder.
976 |
977 |         Args:
978 |             inputs_embeds: [batch, seq_len, multimodal_hidden_size] — vision tower çıktısı
979 |
980 |         Returns:
981 |             [batch, seq_len, text_hidden_size]

```

DiffusionGemmaMultimodalEmbedder.forward()

Çok modlu soft token gömmelerini dil modeli boyutuna projekte eder.

Args:

inputs_embeds: [batch, seq_len, multimodal_hidden_size] — vision tower çıktısı

Returns:

[batch, seq_len, text_hidden_size]

DiffusionGemmaSelfConditioning

Öz-koşullandırma (self-conditioning) modülü.

Difüzyon decoder'ı, bir önceki gürültü giderme adımından gelen logits'leri soft embedding'e çevirip kendi girdisine ekler. Bu, modelin önceki tahmininden öğrenmesini sağlar (self-conditioning).

Yapı: Gemma4 kapılı MLP (gate/up/down) + pre/post RMSNorm.

Çıkış, decoder girdi gömmelerine eklenir.

```

987 | class DiffusionGemmaSelfConditioning(nn.Module):
988 |     """
989 |     Öz-koşullandırma (self-conditioning) modülü.
990 |
991 |     Difüzyon decoder'ı, bir önceki gürültü giderme adımından gelen logits'leri
992 |     soft embedding'e çevirip kendi girdisine ekler. Bu, modelin önceki tahmininden
993 |     öğrenmesini sağlar (self-conditioning).
994 |
995 |     Yapı: Gemma4 kapılı MLP (gate/up/down) + pre/post RMSNorm.
996 |     Çıkış, decoder girdi gömmelerine eklenir.
997 |     """
998 |     def __init__(self, config: DiffusionGemmaTextConfig):
999 |         super().__init__()
1000 |         hidden_size = config.hidden_size
1001 |         intermediate_size = config.intermediate_size
1002 |
1003 |         self.pre_norm = DiffusionGemmaRMSNorm(hidden_size, eps=config.rms_norm_eps)
1004 |         self.post_norm = DiffusionGemmaRMSNorm(hidden_size, eps=config.rms_norm_eps, with
1005 |         self.gate_proj = nn.Linear(hidden_size, intermediate_size, bias=False)
1006 |         self.up_proj = nn.Linear(hidden_size, intermediate_size, bias=False)
1007 |         self.down_proj = nn.Linear(intermediate_size, hidden_size, bias=False)
1008 |         self.act_fn = ACT2FN[config.hidden_activation]
1009 |
1010 |     def forward(self, inputs_embeds, self_conditioning_signal: torch.Tensor) -> torch.Ten
1011 |         """
1012 |         Önceki adımın soft embedding'ini işleyip decoder girdisine ekler.
1013 |
1014 |         Args:
1015 |             inputs_embeds: [batch, canvas_len, hidden_size] — tuval token gömmeleri
1016 |             self_conditioning_signal: [batch, canvas_len, hidden_size] — önceki adım logi
1017 |                 türetilmiş soft embedding (ilk adımda sıfır vektör)
1018 |

```

DiffusionGemmaSelfConditioning.forward()

Önceki adımın soft embedding'ini işleyip decoder girdisine ekler.

Args:

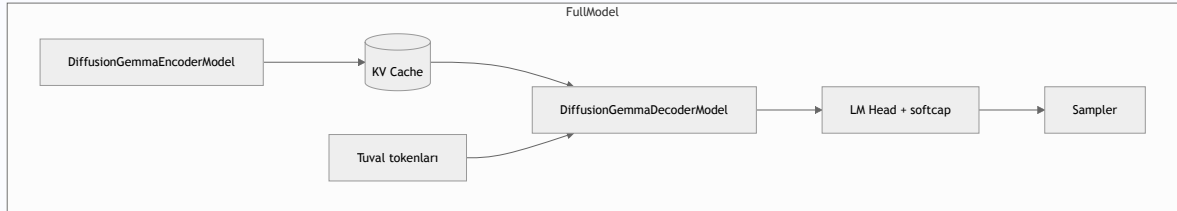
inputs_embeds: [batch, canvas_len, hidden_size] — tuval token gömmeleri

self_conditioning_signal: [batch, canvas_len, hidden_size] — önceki adım logits'inden türetilmiş soft embedding (ilk adımda sıfır vektör)

Returns:

[batch, canvas_len, hidden_size] — öz-koşullandırılmış girdi gömmeleri

5.6 Tam modeller



DiffusionGemmaEncoderTextModel

DiffusionGemma metin encoder gövdesi (saf dil modeli katmanları).

Prompt/context tokenlarını işler, her katmanda KV önbelleğini günceller.

Görüntü girdisi yoktur; çok modlu birleştirme DiffusionGemmaEncoderModel'de yapılır.

Bileşenler: embed_tokens, N encoder katmanı, final RMSNorm, RoPE.

```

1028 | class DiffusionGemmaEncoderTextModel(DiffusionGemmaPreTrainedModel):
1029 |     """
1030 |         DiffusionGemma metin encoder gövdesi (saf dil modeli katmanları).
1031 |
1032 |         Prompt/context tokenlarını işler, her katmanda KV önbelleğini günceller.
1033 |         Görüntü girdisi yoktur; çok modlu birleştirme DiffusionGemmaEncoderModel'de yapılır.
1034 |
1035 |         Bileşenler: embed_tokens, N encoder katmanı, final RMSNorm, RoPE.
1036 |     """
1037 |     config: DiffusionGemmaTextConfig
1038 |     input_modalities = ("text",)
1039 |     _can_record_outputs = {
1040 |         "router_logits": OutputRecorder(DiffusionGemmaTextRouter, index=0),
1041 |         "hidden_states": DiffusionGemmaEncoderTextLayer,
1042 |         "attentions": DiffusionGemmaEncoderTextAttention,
1043 |     }
1044 |
1045 |     def __init__(self, config: DiffusionGemmaTextConfig):
1046 |         super().__init__(config)
1047 |         self.padding_idx = config.pad_token_id
1048 |         self.vocab_size = config.vocab_size
1049 |
1050 |         # bfloat16 dönüşümü embed_scale'i hafif değiştirir; bkz. HF PR #29402
1051 |         self.embed_tokens = DiffusionGemmaTextScaledWordEmbedding(
1052 |             config.vocab_size, config.hidden_size, self.padding_idx, embed_scale=self.con
1053 |         )
1054 |         self.layers = nn.ModuleList(
1055 |             [DiffusionGemmaEncoderTextLayer(config, layer_idx) for layer_idx in range(con
1056 |         )
1057 |         self.norm = DiffusionGemmaRMSNorm(config.hidden_size, eps=config.rms_norm_eps)
1058 |         self.rotary_emb = DiffusionGemmaTextRotaryEmbedding(config)
1059 |         self.unique_layer_types = set(config.layer_types)

```

DiffusionGemmaEncoderTextModel.forward()

Metin encoder'ının tam ileri geçişi: gömme → katmanlar → norm → KV önbelleği.

Args:

input_ids: [batch, seq_len] — token kimlikleri (inputs_embeds ile birlikte verilemez)

attention_mask: padding/causal mask veya önceden hazırlanmış dict

position_ids: [batch, seq_len] — mutlak pozisyon kimlikleri

`past_key_values`: mevcut KV önbelleği (yoksa `DynamicCache` oluşturulur)
`inputs_embeds`: `[batch, seq_len, hidden_size]` — doğrudan gömme girdisi

Returns:

`BaseModelOutputWithPast`: `last_hidden_state` ve güncellenmiş `past_key_values`

DiffusionGemmaEncoderModel

DiffusionGemma encoder modeli — metin + görüntü birleştirme ve KV önbellesi üretimi.

Prompt tokenlarını (ve isteğe bağlı görüntü patch'lerini) işleyerek decoder'ın kullanacağı KV önbellesini oluşturur. Vision tower çıktıları, placeholder tokenların yerine scatter edilir.

Alt bileşenler:

- language_model: saf metin encoder gövdesi
- vision_tower: görüntü özellik çıkarıcı
- embed_vision: görüntü özelliklerini dil uzayına projekte eder

```

1161 | class DiffusionGemmaEncoderModel(DiffusionGemmaPreTrainedModel):
1162 |     """
1163 |         DiffusionGemma encoder modeli — metin + görüntü birleştirme ve KV önbellesi üretimi.
1164 |
1165 |         Prompt tokenlarını (ve isteğe bağlı görüntü patch'lerini) işleyerek decoder'ın
1166 |         kullanacağı KV önbellesini oluşturur. Vision tower çıktıları, placeholder
1167 |         tokenların yerine scatter edilir.
1168 |
1169 |         Alt bileşenler:
1170 |         - language_model: saf metin encoder gövdesi
1171 |         - vision_tower: görüntü özellik çıkarıcı
1172 |         - embed_vision: görüntü özelliklerini dil uzayına projekte eder
1173 |     """
1174 |     # logits/labels filtrelenir; kayıp num_items_in_batch'e bölünmez
1175 |     accepts_loss_kwargs = False
1176 |     config: DiffusionGemmaConfig
1177 |     _can_record_outputs = {
1178 |         "router_logits": OutputRecorder(DiffusionGemmaTextRouter, index=0),
1179 |         "hidden_states": DiffusionGemmaEncoderTextLayer,
1180 |         "attentions": DiffusionGemmaEncoderTextAttention,
1181 |     }
1182 |
1183 |     def __init__(self, config: DiffusionGemmaConfig):
1184 |         super().__init__(config)
1185 |         self.vocab_size = config.text_config.vocab_size
1186 |
1187 |         self.language_model = DiffusionGemmaEncoderTextModel(config=config.text_config)
1188 |         self.vision_tower = AutoModel.from_config(config.vision_config)
1189 |         self.embed_vision = DiffusionGemmaMultimodalEmbedder(config.vision_config, config
1190 |
1191 |         # Ağırlıkları başlat ve son işleme uygula
1192 |         self.post_init()

```

DiffusionGemmaEncoderModel.forward()

Encoder'ın tam ileri geçişi: metin+görüntü birleştirme ve KV önbellesi üretimi.

Args:

input_ids: [batch, seq_len] — prompt token kimlikleri

pixel_values: [batch, C, H, W] — görüntü piksel değerleri (opsiyonel)

attention_mask: encoder attention mask'i

position_ids: token pozisyon kimlikleri

past_key_values: mevcut KV önbelleği

mm_token_type_ids: çok modlu token tip kimlikleri (görüntü mask için)

inputs_embeds: doğrudan gömme girdisi (input_ids ile birlikte verilemez)

image_position_ids: [batch, max_patches, 2] — görüntü patch koordinatları

Returns:

BaseModelOutputWithPast: encoder çıktısı ve güncellenmiş KV önbelleği

DiffusionGemmaDecoderModel

DiffusionGemma decoder modeli — tuval gürültü giderme ve encoder çapraz dikkat.

Tuval (canvas) tokenlarını işler: iki yönlü self-attention + encoder KV
önbelleğine salt okunur çapraz dikkat. Decoder KV önbelleğini güncellemez.

EncoderTextModel ile ortak ağırlıkları paylaşır; ek olarak self_conditioning
modülü vardır. Çıktıda past_key_values döndürülmez.

```

1388 | class DiffusionGemmaDecoderModel(DiffusionGemmaPreTrainedModel):
1389 |     """
1390 |         DiffusionGemma decoder modeli — tuval gürültü giderme ve encoder çapraz dikkat.
1391 |
1392 |         Tuval (canvas) tokenlarını işler: iki yönlü self-attention + encoder KV
1393 |         önbelleğine salt okunur çapraz dikkat. Decoder KV önbelleğini güncellemez.
1394 |
1395 |         EncoderTextModel ile ortak ağırlıkları paylaşır; ek olarak self_conditioning
1396 |         modülü vardır. Çıktıda past_key_values döndürülmez.
1397 |     """
1398 |     config: DiffusionGemmaConfig
1399 |     input_modalities = ("text",)
1400 |     _can_record_outputs = {
1401 |         "router_logits": OutputRecorder(DiffusionGemmaTextRouter, index=0),
1402 |         "hidden_states": DiffusionGemmaDecoderTextLayer,
1403 |         "attentions": DiffusionGemmaDecoderTextAttention,
1404 |     }
1405 |
1406 |     def __init__(self, config: DiffusionGemmaConfig):
1407 |         super().__init__(config)
1408 |         self.text_config = config.text_config
1409 |         self.padding_idx = config.text_config.pad_token_id
1410 |         self.vocab_size = config.text_config.vocab_size
1411 |
1412 |         self.embed_tokens = DiffusionGemmaTextScaledWordEmbedding(
1413 |             num_embeddings=config.text_config.vocab_size,
1414 |             embedding_dim=config.text_config.hidden_size,
1415 |             padding_idx=self.padding_idx,
1416 |             embed_scale=config.text_config.hidden_size**0.5,
1417 |         )
1418 |         self.layers = nn.ModuleList(
1419 |             [

```

DiffusionGemmaDecoderModel.forward()

Decoder'ın tam ileri geçişi: tuval gömme → öz-koşullandırma → katmanlar → norm.

Args:

decoder_input_ids: [batch, canvas_len] — tuval token kimlikleri

past_key_values: encoder'ın salt okunur KV önbelleği (zorunlu)

self_conditioning_logits: [batch, canvas_len, vocab_size] — önceki adım logits'leri
self_conditioning_mask: [batch] — örnek bazında öz-koşullandırma açma/kapama
decoder_attention_mask: encoder önbelleği + tuval için attention mask
decoder_position_ids: [batch, canvas_len] — tuval pozisyon kimlikleri

Returns:

BaseModelOutput: tuval son katman gizli durumları (KV önbelleği döndürülmez)

DiffusionGemmaModel

DiffusionGemma tam modeli — encoder + difüzyon decoder.

İki aşamalı mimari:

1. Encoder (DiffusionGemmaEncoderModel): prompt/context'i işler, KV önbelleği üretir. Gemma4Model'e çok benzer; görüntü desteği vardır.
2. Decoder (DiffusionGemmaDecoderModel): tuval tokenlarını gürültü giderme ile işler.

Klasik encoder-decoder'dan kritik fark:

Encoder, decoder'a gizli durumları değil yalnızca KV önbelleğini verir.

Decoder bu önbelleği salt okunur kullanır (çapraz dikkat).

Ağırlık paylaşımı: encoder ve decoder metin katmanları ortak ağırlıkları paylaşır;

self_conditioning yalnızca decoder'da bulunur.

```

1662 | class DiffusionGemmaModel(DiffusionGemmaPreTrainedModel):
1663 |     """
1664 |         DiffusionGemma tam modeli — encoder + difüzyon decoder.
1665 |
1666 |         İki aşamalı mimari:
1667 |         1. Encoder (DiffusionGemmaEncoderModel): prompt/context'i işler, KV önbelleği üretir.
1668 |             Gemma4Model'e çok benzer; görüntü desteği vardır.
1669 |         2. Decoder (DiffusionGemmaDecoderModel): tuval tokenlarını gürültü giderme ile işler.
1670 |
1671 |         Klasik encoder-decoder'dan kritik fark:
1672 |         Encoder, decoder'a gizli durumları değil yalnızca KV önbelleğini verir.
1673 |         Decoder bu önbelleği salt okunur kullanır (çapraz dikkat).
1674 |
1675 |         Ağırlık paylaşımı: encoder ve decoder metin katmanları ortak ağırlıkları paylaşır;
1676 |         self_conditioning yalnızca decoder'da bulunur.
1677 |     """
1678 |     # Encoder metin ağırlıkları decoder'da da var; self-conditioning yalnızca decoder'da
1679 |     _tied_weights_keys = {
1680 |         "encoder.language_model.norm.weight": "decoder.norm.weight",
1681 |         # Aşağıdaki satırlar katman ağırlıklarını bağlar; buffer'lar (layer_sclar vb.) b
1682 |         r"encoder.language_model.layers\.(?:[^\.]+\.)*weight": r"decoder.layers\.(?:[^\.]+\.)",
1683 |         r"encoder.language_model.layers\.(?:[^\.]+\.)*scale": r"decoder.layers\.(?:[^\.]+\.)",
1684 |         r"encoder.language_model.layers\.(?:[^\.]+\.)*per_expert_scale": r"decoder.layers\.",
1685 |         r"encoder.language_model.layers\.(?:[^\.]+\.)*gate_up_proj": r"decoder.layers\.(?:[^\.]+\.)",
1686 |         r"encoder.language_model.layers\.(?:[^\.]+\.)*down_proj": r"decoder.layers\.(?:[^\.]+\.)",
1687 |         "encoder.language_model.embed_tokens.weight": "decoder.embed_tokens.weight",
1688 |     }
1689 |
1690 |     def __init__(self, config: DiffusionGemmaConfig):
1691 |         super().__init__(config)
1692 |
1693 |         self.encoder = DiffusionGemmaEncoderModel(config)

```

DiffusionGemmaModel.forward()

DiffusionGemma tam model ileri geiři: encode → decode.

İki aşama:

1. Encoder: input_ids varsa prompt'u KV önbelleğine yazar.
Yoksa mevcut past_key_values kullanılır (en az biri zorunlu).
2. Decoder: tuval tokenlarını işler; encoder KV önbelleğine çapraz dikkat + tuval içi iki yönlü self-attention uygular.

decoder_input_ids verilmezse tuval, vocab'tan uniform rastgele örneklenir (difüzyon başlangıç durumu).

Args:

input_ids: [batch, seq_len] — encode edilecek prompt tokenları
attention_mask: encoder attention mask'i
past_key_values: mevcut KV önbelleği
position_ids: encoder pozisyon kimlikleri
decoder_input_ids: [batch, canvas_len] — tuval tokenları
self_conditioning_logits: önceki adım logits'leri
self_conditioning_mask: örnek bazında öz-koşullandırma maskesi
decoder_attention_mask: decoder attention mask'i
decoder_position_ids: tuval pozisyon kimlikleri

Returns:

DiffusionGemmaModelOutputWithPast: decoder çıktısı + encoder KV önbelleği

DiffusionGemmaForBlockDiffusion

Blok difüzyon için DiffusionGemma dil modeli başlığı (LM Head).

DiffusionGemmaModel'i çağırarak prompt KV önbelleği koşullu tuval gizli durumlarını üretir; lm_head ile vocab logits'lerine dönüştürür.

Blok difüzyon döngüsü:

1. Encoder prompt'u KV önbelleğine yazar
2. Decoder gürültülü tuvali işler (öz-koşullandırma ile)
3. lm_head logits üretir → softcapping uygulanır
4. Logits bir sonraki gürültü giderme adımında self_conditioning_logits olarak decoder'a geri beslenir

lm_head ağırlıkları decoder embed_tokens ile tied (paylaşımlı).

```

1792 | class DiffusionGemmaForBlockDiffusion(DiffusionGemmaPreTrainedModel, DiffusionGemmaGenera
1793 |     """
1794 |     Blok difüzyon için DiffusionGemma dil modeli başlığı (LM Head).
1795 |
1796 |     DiffusionGemmaModel'i çağırarak prompt KV önbelleği koşullu tuval gizli
1797 |     durumlarını üretir; lm_head ile vocab logits'lerine dönüştürür.
1798 |
1799 |     Blok difüzyon döngüsü:
1800 |         1. Encoder prompt'u KV önbelleğine yazar
1801 |         2. Decoder gürültülü tuvali işler (öz-koşullandırma ile)
1802 |         3. lm_head logits üretir → softcapping uygulanır
1803 |         4. Logits bir sonraki gürültü giderme adımında self_conditioning_logits olarak
1804 |            decoder'a geri beslenir
1805 |
1806 |     lm_head ağırlıkları decoder embed_tokens ile tied (paylaşımlı).
1807 |     """
1808 |     base_model_prefix = "model"
1809 |     _tied_weights_keys = {"lm_head.weight": "model.decoder.embed_tokens.weight"}
1810 |     generation_config_class = DiffusionGemmaGenerationConfig
1811 |
1812 |     def __init__(self, config: DiffusionGemmaConfig):
1813 |         super().__init__(config)
1814 |
1815 |         self.model = DiffusionGemmaModel(config)
1816 |         self.lm_head = nn.Linear(config.text_config.hidden_size, config.text_config.vocab
1817 |         self.final_logit_softcapping = config.text_config.final_logit_softcapping
1818 |
1819 |         # Ağırlıkları başlat ve son işleme uygula
1820 |         self.post_init()
1821 |
1822 |     def get_input_embeddings(self):
1823 |         return self.model.encoder.language_model.get_input_embeddings()

```

DiffusionGemmaForBlockDiffusion.forward()

Blok difüzyon ileri geçişi: model → lm_head → softcapped logits.

Args:

input_ids: [batch, seq_len] — prompt tokenları

attention_mask: encoder mask

past_key_values: KV önbelleği

position_ids: encoder pozisyon kimlikleri

decoder_input_ids: [batch, canvas_len] — tuval tokenları

self_conditioning_logits: önceki adım logits'leri

self_conditioning_mask: öz-koşullandırma maskesi

decoder_attention_mask: decoder mask

decoder_position_ids: tuval pozisyon kimlikleri

Returns:

DiffusionGemmaBlockDiffusionOutputWithPast: softcapped logits ve model çıktıları

5.7 Yardımcılar

get_block_sequence_ids_for_mask

Çok modlu attention mask için görüntü blok kimliklerini hesaplar.

Vision tokenları (tip 1 veya 2) ardışık gruplara ayrılır; her grup bir blok kimliği alır. Metin tokenları -1 ile işaretlenir (blok dışı).

Args:

mm_token_type_ids: [batch, seq_len] — 0=metin, 1/2=görüntü token tipi

device: hedef cihaz

Returns:

[batch, seq_len] — blok kimlikleri (-1 metin, >=0 görüntü blokları)

```

1136 | def get_block_sequence_ids_for_mask(mm_token_type_ids: torch.Tensor, device: torch.device
1137 |     """
1138 |     Çok modlu attention mask için görüntü blok kimliklerini hesaplar.
1139 |
1140 |     Vision tokenları (tip 1 veya 2) ardışık gruplara ayrılır; her grup bir blok
1141 |     kimliği alır. Metin tokenları -1 ile işaretlenir (blok dışı).
1142 |
1143 |     Args:
1144 |         mm_token_type_ids: [batch, seq_len] — 0=metin, 1/2=görüntü token tipi
1145 |         device: hedef cihaz
1146 |
1147 |     Returns:
1148 |         [batch, seq_len] — blok kimlikleri (-1 metin, >=0 görüntü blokları)
1149 |     """
1150 |     mm_token_type_ids = mm_token_type_ids.to(device)
1151 |
1152 |     is_vision = (mm_token_type_ids == 1) | (mm_token_type_ids == 2)
1153 |     is_prev_vision = torch.roll(is_vision, shifts=1, dims=-1)
1154 |     is_prev_vision[..., 0] = False
1155 |     new_vision_starts = is_vision & ~is_prev_vision
1156 |     vision_group_ids = torch.cumsum(new_vision_starts.int(), dim=-1) - 1
1157 |     block_sequence_ids = torch.where(is_vision, vision_group_ids, -1)
1158 |     return block_sequence_ids
1159 |
1160 |
1161 | class DiffusionGemmaEncoderModel(DiffusionGemmaPreTrainedModel):
1162 |     """
1163 |     DiffusionGemma encoder modeli — metin + görüntü birleştirme ve KV ön belleği üretimi.
1164 |
1165 |     Prompt tokenlarını (ve isteğe bağlı görüntü patch'lerini) işleyerek decoder'ın
1166 |     kullanacağı KV ön belleğini oluşturur. Vision tower çıktıları, placeholder
1167 |     tokenların yerine scatter edilir.

```

DiffusionGemmaModelOutputWithPast

DiffusionGemmaModel çıktı konteyneri.

Standart BaseModelOutputWithPast alanlarına ek olarak encoder son katman gizli durumunu taşır (eğitimde encoder kaybı hesaplamak için).

Alanlar:

last_hidden_state: decoder tuval çıktısı

past_key_values: encoder KV önbelleği

hidden_states: ara katman durumları (opsiyonel)

attentions: attention ağırlıkları (opsiyonel)

encoder_last_hidden_state: encoder son katman çıktısı (input_ids verilmişse)

```

1625 | class DiffusionGemmaModelOutputWithPast(BaseModelOutputWithPast):
1626 |     """
1627 |         DiffusionGemmaModel çıktı konteyneri.
1628 |
1629 |         Standart BaseModelOutputWithPast alanlarına ek olarak encoder son katman
1630 |         gizli durumunu taşır (eğitimde encoder kaybı hesaplamak için).
1631 |
1632 |         Alanlar:
1633 |             last_hidden_state: decoder tuval çıktısı
1634 |             past_key_values: encoder KV önbelleği
1635 |             hidden_states: ara katman durumları (opsiyonel)
1636 |             attentions: attention ağırlıkları (opsiyonel)
1637 |             encoder_last_hidden_state: encoder son katman çıktısı (input_ids verilmişse)
1638 |     """
1639 |
1640 |     encoder_last_hidden_state: torch.FloatTensor | None = None
1641 |
1642 |
1643 | @dataclass
1644 | class DiffusionGemmaBlockDiffusionOutputWithPast(CausalLMOutputWithPast):
1645 |     """
1646 |         Blok difüzyon dil modeli çıktı konteyneri.
1647 |
1648 |         Tuval üzerindeki logits, gizli durumlar ve encoder çıktısını bir arada döner.
1649 |
1650 |         Alanlar:
1651 |             loss: dil modelleme kaybı (opsiyonel)
1652 |             logits: [batch, canvas_len, vocab_size] - softcapping öncesi skorlar
1653 |             past_key_values: encoder KV önbelleği
1654 |             hidden_states: ara katman durumları (opsiyonel)
1655 |             attentions: attention ağırlıkları (opsiyonel)
1656 |             encoder_last_hidden_state: encoder son katman çıktısı (opsiyonel)

```


DiffusionGemmaBlockDiffusionOutputWithPast

Blok difüzyon dil modeli çıktı konteyneri.

Tuval üzerindeki logits, gizli durumlar ve encoder çıktısını bir arada döner.

Alanlar:

loss: dil modelleme kaybı (opsiyonel)

logits: [batch, canvas_len, vocab_size] — softcapping öncesi skorlar

past_key_values: encoder KV önbelleği

hidden_states: ara katman durumları (opsiyonel)

attentions: attention ağırlıkları (opsiyonel)

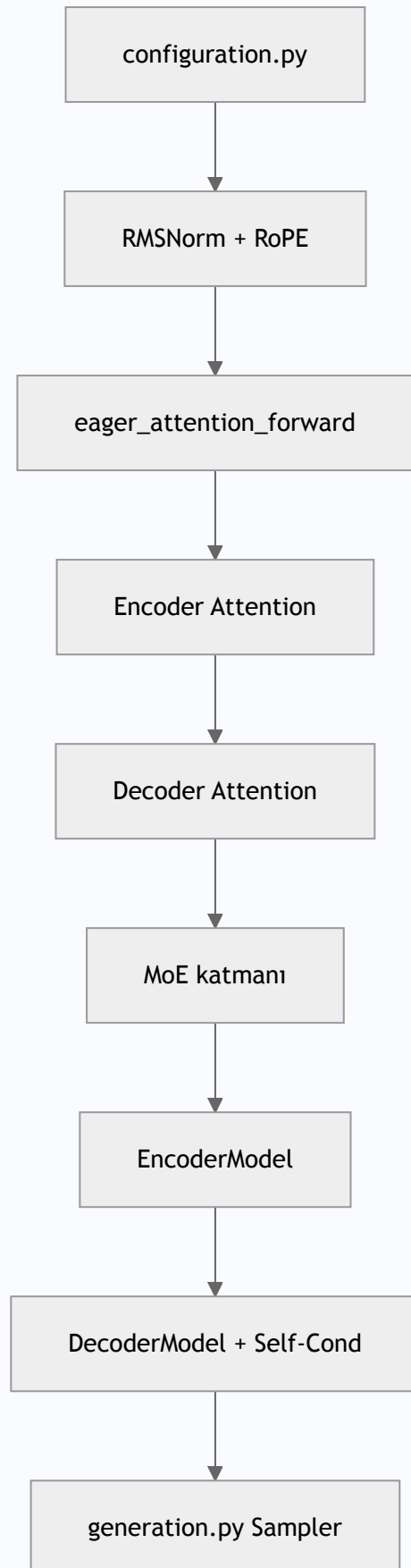
encoder_last_hidden_state: encoder son katman çıktısı (opsiyonel)

```

1644 | class DiffusionGemmaBlockDiffusionOutputWithPast(CausalLMOutputWithPast):
1645 |     """
1646 |     Blok difüzyon dil modeli çıktı konteyneri.
1647 |
1648 |     Tuval üzerindeki logits, gizli durumlar ve encoder çıktısını bir arada döner.
1649 |
1650 |     Alanlar:
1651 |         loss: dil modelleme kaybı (opsiyonel)
1652 |         logits: [batch, canvas_len, vocab_size] — softcapping öncesi skorlar
1653 |         past_key_values: encoder KV önbelleği
1654 |         hidden_states: ara katman durumları (opsiyonel)
1655 |         attentions: attention ağırlıkları (opsiyonel)
1656 |         encoder_last_hidden_state: encoder son katman çıktısı (opsiyonel)
1657 |     """
1658 |
1659 |     encoder_last_hidden_state: torch.FloatTensor | None = None
1660 |
1661 |
1662 | class DiffusionGemmaModel(DiffusionGemmaPreTrainedModel):
1663 |     """
1664 |     DiffusionGemma tam modeli — encoder + difüzyon decoder.
1665 |
1666 |     İki aşamalı mimari:
1667 |         1. Encoder (DiffusionGemmaEncoderModel): prompt/context'i işler, KV önbelleği üreti
1668 |             Gemma4Model'e çok benzer; görüntü desteği vardır.
1669 |         2. Decoder (DiffusionGemmaDecoderModel): tuval tokenlarını gürültü giderme ile işle
1670 |
1671 |     Klasik encoder-decoder'dan kritik fark:
1672 |         Encoder, decoder'a gizli durumları değil yalnızca KV önbelleğini verir.
1673 |         Decoder bu önbelleği salt okunur kullanır (çapraz dikkat).
1674 |
1675 |     Ağırlık paylaşımı: encoder ve decoder metin katmanları ortak ağırlıkları paylaşır;

```

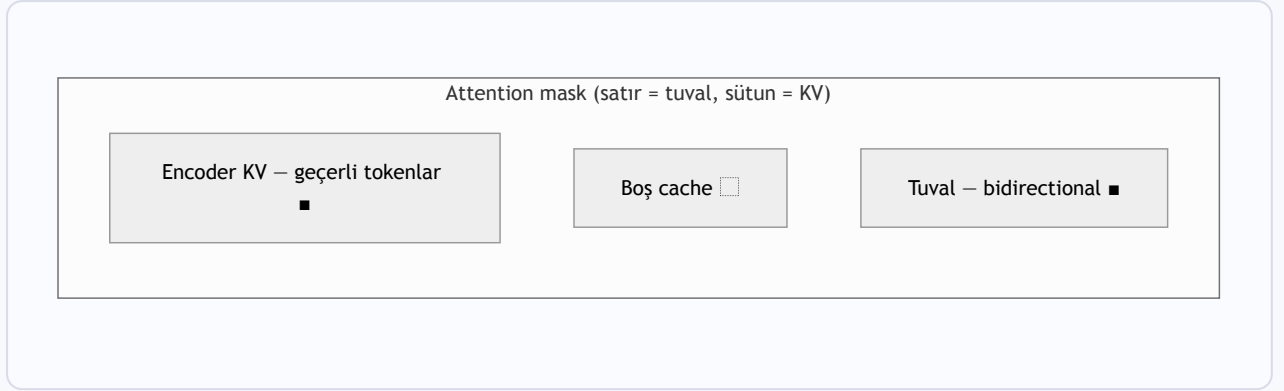
6. Öğrenme Rehberi



1. **configuration.py** — boyutları öğren
2. **Temel bloklar** — RMSNorm, RoPE, GQA, attention
3. **Encoder vs Decoder attention** — KV cache farkı
4. **MoE** — dense FFN + router + experts
5. **Tam model** — encoder → cache → decoder → lm_head
6. **generation.py** — entropi sampler döngüsü

Decoder attention mask yapısı

Tuval tokenları encoder cache'teki geçerli pozisyonlara ve birbirlerine bakabilir:



Kaynaklar

- HuggingFace: `modeling_diffusion_gemma.py`
- EntropyBoundSampler: [arXiv 2505.24857](https://arxiv.org/abs/2505.24857)
- github.com/g-hano/paper-implementations